Predictive-Proactive Congestion Avoidance for Cross-Domain Data Center Interconnect

Xue Zhang, Yuming Zhang, *Member, IEEE*, Wei Quan, *Senior Member, IEEE*, Mingyuan Liu, *Member, IEEE*, Yirong Zhuang, Yan Luo, Tong Luo, Jie Liang, Kang Liu, and Hongke Zhang

***Abstract*—The inherent cross-domain nature of data center interconnect (DCI) presents a critical challenge in satisfying the transmission requirements of intelligent computing services such as ChatGPT and DeepSeek. However, existing transmission control solutions often suffer from inaccurate network state analysis and limited real-time adaptability to dynamic cross-domain traffic, resulting in low resource utilization and poor performance. To this end, this paper proposes a Predictive and Proactive Congestion Avoidance mechanism (PPCA) for DCI. Firstly, a digital twin network (DTN)-inspired congestion avoidance framework is proposed to support virtual mirroring of physical traffic states and efficient closed-loop traffic management, thereby enhancing the proactivity and stability of network operations. Then, a neighbor-assisted traffic prediction method is developed based on the temporal convolutional network (TCN) model. It integrates neighbor information to capture spatiotemporal dependencies effectively, enabling precise virtual mirroring of traffic states. Furthermore, an exploration-exploitation-based flow scheduling algorithm is designed to generate the optimal scheduling strategy for the predicted traffic, thereby avoiding congestion. Finally, extensive simulations are conducted with real-world traffic replaying, and results demonstrate that PPCA achieves superior abilities across 5 classical baselines in mitigating network congestion, balancing traffic load, and handling burst traffic.**

***Index Terms*—Data center interconnect, congestion avoidance, traffic prediction, digital twin network, cross-domain transmission.**

I. Introduction

THE widespread adoption of artificial intelligence (AI), such as ChatGPT and DeepSeek, has sharply increased computing demands across data center interconnect (DCI) [1]. The bandwidth capacity and transmission stability of DCI directly determine the efficiency of inter–data center computing tasks [2]. Consequently, enhancing DCI transmission performance has become a key research focus. However, the inherently cross-domain and heterogeneous nature of DCI transmission poses significant challenges to achieving stable data delivery. As illustrated in Fig. 1, the DCI architecture represents a typical cross-domain transmission scenario, spanning multiple geographic regions and integrating heterogeneous links from diverse network operators. In long-distance, multi-hop cross-domain transmission, bandwidth fluctuations and sudden network congestion become more severe, impacting network scheduling and task optimization. Therefore, developing a transmission solution that dynamically adapts to complex cross-domain transmission environments is crucial for ensuring efficient and stable transmission in DCI.

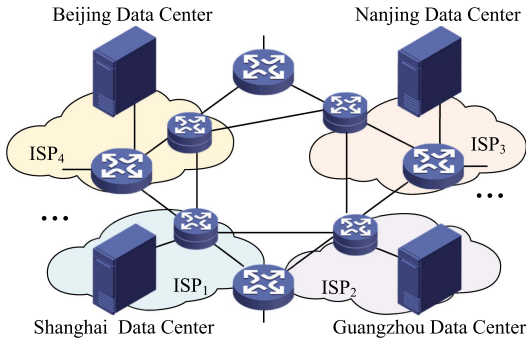


Fig. 1. Overview of transmission scenario for DCI.

Typical DCI transmission mechanisms primarily rely on reactive congestion control methods, such as delay/loss detection [3], [4], [5]. However, cross-domain transmission often experiences high propagation delays and non-congestion-related packet losses, making these indicators unreliable [6], [7]. Furthermore, congestion in such cross-domain environments is influenced by multiple complex factors, and relying solely on packet loss or delay fails to provide a comprehensive assessment. *Consequently, traditional reactive approaches suffer from inaccurate network state analysis and thus fail to meet the requirements of DCI.*

Recent research has shifted toward active transmission optimization solutions to overcome the limitations of traditional

Received 1 May 2025; revised 6 September 2025; accepted 7 October 2025. Date of publication 16 October 2025; date of current version 11 December 2025. This work was supported in part by the National Key R&D Program of China under Grant 2022YFB2901900, in part by the National Natural Science Foundation of China under Grant 62322102, Grant 62372035, and Grant 62402030, in part by the Talent Fund of Beijing Jiaotong University under Grant 2024XKRC063, and in part by the China Postdoctoral Science Foundation under Grant 2024M761544. Recommended for acceptance by Dr. Chunxiao Jiang. *(Corresponding author: Yuming Zhang.)*

Xue Zhang, Yuming Zhang, Wei Quan, Mingyuan Liu, Yan Luo, and Hongke Zhang are with the School of Electronic and Information Engineering, National Engineering Research Center of Advanced Network Technologies, Beijing Jiaotong University, Beijing 100044, China (e-mail: snowzhang@bjtu.edu.cn; ymzhang3@bjtu.edu.cn; weiquan@bjtu.edu.cn; myliu@bjtu.edu.cn; yanluo@bjtu.edu.cn; hkzhang@bjtu.edu.cn).

Yirong Zhuang and Jie Liang are with China Telecom Research Institute, Guangzhou 510630, China (e-mail: zhuangyir@chinatelecom.cn; liangjie6@chinatelecom.cn).

Tong Luo is with China Mobile (Suzhou) Software Technology Corporation, Suzhou 215000, China (e-mail: luotong@bjtu.edu.cn).

Kang Liu is with Peng Cheng Laboratory, Shenzhen 518055, China (e-mail: liuk02@pcl.ac.cn).

Digital Object Identifier 10.1109/TNSE.2025.3622383

2327-4697 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Universitas Indonesia. Downloaded on October 03,2026 at 15:35:47 UTC from IEEE Xplore. Restrictions apply.

reactive control. These solutions typically leverage programmable network technologies to collect network data and dynamically adjust transmission strategies based on the states, thereby enhancing the transmission efficiency [8], [9]. Existing methods can generally be categorized into three types: learning-based policy generation, flow feature mapping-based policy selection, and heuristic algorithm-based transmission optimization. Learning-based approaches emphasize intelligent state analysis and adaptive optimization [10]. However, they typically rely on performance-assisted reward functions rather than directly analyzing the root causes of network performance bottlenecks. This indirect and non-explainable process limits their ability to identify and mitigate bottlenecks precisely. Moreover, learning models typically require substantial computational resources, which constrain their applicability in highly dynamic, real-time cross-domain networks [11], [12]. Flow feature mapping-based approaches aim to balance strategy generation efficiency and transmission performance [13]. Nevertheless, they typically rely on fixed, pre-defined strategy mappings, which struggle to maintain optimal performance when encountering bursty or unseen traffic patterns. Heuristic-based approaches, while offering low complexity and explainable solutions, primarily rely on transmission performance metrics and lack fine-grained analysis of network state, thereby restricting their efficiency in highly dynamic network environments [14]. *In addition, the collection of network data inevitably requires time, and decisions based on delayed information may reduce the effectiveness of cross-domain transmission optimization.*

Digital twin network (DTN) offers a promising paradigm for transmission optimization [15], [16], [17]. The core idea of DTN is to construct virtual mirroring of the physical network to perform predictive analysis, enabling simulation and evaluation of transmission strategies to proactively avoid potential bottlenecks. Inspired by this concept, we design a Predictive and Proactive Congestion Avoidance mechanism (PPCA) for dynamic traffic management in DCI. PPCA employs a neighbor-assisted traffic prediction method to capture long-term dependencies and local spatial correlations of cross-domain traffic. It also integrates an exploration-exploitation-based flow scheduling algorithm, which can generate transmission strategies for the predicted traffic. These strategies are then deployed on switches, enabling active scheduling of DCI traffic. In this way, PPCA translates the DTN concept into a congestion avoidance framework, allowing proactive traffic management in cross-domain, highly dynamic environments. The main contributions of this paper are as follows:

- This paper proposes a novel predictive and proactive congestion avoidance framework for traffic management in DCI. Inspired by DTN, the framework integrates an intelligent module that enables the virtual mirroring of physical traffic states, allowing accurate analysis and prediction of network conditions. Based on the predictions, the framework conducts closed-loop, proactive traffic management to achieve adaptive congestion avoidance.
- This paper develops a neighbor-assisted traffic prediction method. Specifically, the method builds customized models based on temporal convolutional network (TCN) for each inter-data center (DC) switch to capture the long-term dependencies of network traffic. Furthermore, the method leverages real-time traffic from adjacent switches to enhance local spatial feature extraction, thereby improving prediction accuracy.
- This paper presents a flow scheduling algorithm for proactive congestion avoidance. Specifically, the algorithm treats per-flow scheduling based on predicted traffic as discrete actions and selects a subset of actions to construct proactive scheduling strategies, evaluated by load-balancing effectiveness. Through iterative action selection and evaluation, the algorithm converges to an optimal strategy. Additionally, an explainable exploration-exploitation mechanism is introduced, enabling parallel strategy assessment and adaptive action combination expansion, accelerating algorithm execution and improving congestion avoidance performance.
- This paper evaluates the performance of PPCA with real-world traffic replay and verifies its effectiveness in terms of prediction accuracy and transmission performance. The results reveal that PPCA outperforms the representative prediction baselines and demonstrates enhanced performance across five flow scheduling baselines in reducing network congestion, balancing traffic load, and handling burst traffic.

The remainder of this paper is organized as follows. Section II presents the related works. Section III shows the system model and problem formulation. The proposed PPCA solution is explained in detail in Section IV. In Section V, we simulate the performance of the proposed PPCA solution. Section VI is the conclusion of this paper.

II. Related Work

This section surveys the related work and summarizes it into categorized groups. A feature comparison between previous methods and the proposed mechanism is presented in Table I. Several representative studies are summarized in Table II. The following provides a detailed introduction to these related works.

In recent years, deep learning techniques have been widely adopted for network traffic prediction, aiming to address the complex spatiotemporal dependencies inherent in traffic data. Among them, long short-term memory (LSTM) networks have emerged as one of the most commonly used approaches in early studies [18], [19], [20]. Alawe et al. apply LSTM to predict traffic load dynamics in 5G networks, supporting adaptive resource provisioning [21]. To alleviate the high computational cost associated with LSTM training, Hua et al. propose a sparse LSTM model to improve efficiency and generalization capability [22]. These studies demonstrate LSTM's strength in capturing temporal dependencies but also highlight challenges related to training cost and parameter overhead. To improve prediction performance, researchers explore hybrid architectures that integrate multiple deep learning networks [23], [24]. Li et al. propose a composite framework that integrates residual neural network (ResNet), LSTM, and attention mechanisms to achieve stable and precise prediction [25]. To further capture the long-term

Authorized licensed use limited to: Universitas Indonesia. Downloaded on October 03,2026 at 15:35:47 UTC from IEEE Xplore. Restrictions apply.

TABLE I
FEATURE COMPARISON BETWEEN PREVIOUS APPROACHES AND OUR PROPOSED MECHANISM

Feature	Traffic Prediction Approaches	Congestion Control Approaches	Learning-Based Approaches	Policy Mapping Approaches	Heuristic-Based Approaches	PPCA(ours)
Network status analysis	✓	✓	✓	✓	✗	✓
Network status prediction	✓	✗	✗	✗	✗	✓
Dynamic transmission optimization	✗	✓	✓	✓	✓	✓
Low deployment overhead	✗	✓	✗	✓	✓	✓
Network proactive management	✗	✗	✗	✗	✗	✓
Handling bursty traffic	✗	✗	✓	✗	✗	✓
Explainability	✗	✓	✗	✓	✓	✓
Low complexity	✗	✓	✗	✓	✓	✓

TABLE II
COMPARISON AMONG DIFFERENT METHODS FOR CONGESTION AVOIDANCE IN CROSS-DOMAIN NETWORK

Methods	Characteristics	Limitations
[43]	Uses deep reinforcement learning (DRL) integrated with local search algorithm for real-time routing optimization.	Using measured traffic as input may cause the strategy to lag behind the network state.
[44]	Uses neural networks to capture the spatiotemporal dependencies of traffic for real-time routing optimization.	Lacks proactive scheduling capability, and its black-box decisions offer little insight for further optimization.
[47]	Uses an improved DDPG algorithm integrated with Dijkstra algorithm for real-time routing optimization.	Reactive routing adjustments limit real-time adaptability and incur high computational overhead.
[52], [53]	Trains a deep neural network to map historical traffic to routing configurations.	Struggles to handle bursty traffic and relies on fixed strategies, lacking the flexibility required for fine-grained optimization.
[54], [55]	Design a machine learning-based traffic classifier for online routing decisions.	Relies solely on statistical features for classification, ignoring the temporal feature of traffic.
[58]	Employs greedy heuristics and linear programming to achieve high-availability traffic engineering.	Disregards traffic feature, resulting in limited adaptability to dynamic traffic conditions.
[59]	Uses the AVRO algorithm to realize SDN routing optimization.	Relies on reactive scheduling, which is delayed in responding to sudden congestion events.
PPCA(ours)	A neighbor-assisted traffic prediction method capturing the spatiotemporal dependencies of traffic, and an exploration-exploitation-based flow scheduling algorithm performing congestion avoidance based on the predicted traffic.	Requires adaptive parameter tuning.

dependencies of network traffic, Bi et al. propose a hybrid traffic prediction method based on TCN, LSTM, and Savitzky-Golay, which deal with the instability and nonlinearity of network traffic [26]. These hybrid models have shown superior prediction accuracy and robustness, making them a prevailing direction in recent research. The Transformer architecture also garners increasing attention. Liu et al. and Hu et al. propose transformer-based models for cellular traffic prediction, which are effective in capturing both periodic and short-term dependencies [27], [28]. These studies highlight the potential of attention mechanisms in enhancing model expressiveness and identifying key temporal patterns [29]. In addition to general-purpose models, several works have proposed customized solutions tailored to specific application scenarios [30], [31]. Nie et al. investigate the issues of industrial Internet of Things-oriented backbone network traffic prediction and propose an effective prediction method using multi-task learning, which leverages link loads as additional information to improve prediction accuracy [32]. Wang et al. propose a prediction model relying on temporal and relational information to forecast the capacity needed by high-priority inter-DC traffic [33]. Regarding the extraction of spatial features in network traffic, Chen et al. propose an end-to-end model based on graph convolutional network (GCN) and LSTM for dynamic network link prediction [34]. Xie et al. propose a multi-range multi-level spatiotemporal learning based network traffic prediction model [35]. By leveraging graph convolution and 1D-dilated convolution, the model can capture spatiotemporal dependencies. *However, the above studies only achieve traffic prediction without applying it to transmission optimization.*

Traditional congestion control methods typically rely on network feedback signals to dynamically adjust transmission behaviors [36], [37]. Geng et al. propose a delay-based congestion control scheme that utilizes queue depth monitoring [38]. Long et al. propose a link-segmentation congestion control algorithm to provide timely congestion feedback [39]. Similar to DCI, wide area network is also used for network transmission across multiple geographic regions [40]. Quan et al. propose a cybertwin-driven learning-based transmission mechanism to adjust transmission control methods for efficient transmission adaptively [41]. *Although these approaches offer advantages in terms of low deployment overhead and practical applicability, their reactive nature results in imprecise congestion awareness and restricts adaptability in DCI.*

Driven by the progress of reinforcement learning (RL) and GCN, an increasing number of studies explore learning-based traffic scheduling methods for transmission optimization [42], [43], [44], [45], [46]. Chen et al. propose ALBRL, a software defined network (SDN)-based traffic scheduling architecture leveraging improved deep deterministic policy gradient

Authorized licensed use limited to: Universitas Indonesia. Downloaded on October 03,2026 at 15:35:47 UTC from IEEE Xplore. Restrictions apply.

(DDPG) and Dijkstra algorithm to achieve fast transmission optimization [47]. Liu et al. propose a deadline-constrained multi-agent collaborative transmission mechanism, which achieved stable flow scheduling in congestion scenarios [48]. *Although these approaches perform well, learning-based solutions suffer from poor scalability and limited real-time capability in highly dynamic and complex scenarios due to their high computational and memory overhead [49], [50].*

To reduce the complexity of real-time scheduling, some research has focused on policy mapping approaches based on flow features [51], [52]. Perry et al. train deep neural networks to map historical traffic patterns directly to optimal routing configurations, improving policy stability [53]. Liu et al. construct candidate routing sets via traffic matrix clustering and employ lightweight classifiers to make online routing decisions, reducing computational overhead [54], [55]. *While these approaches strike a balance between efficiency and performance, they tend to suffer from degraded accuracy and robustness when handling bursty or unpredictable traffic patterns.*

Other efforts explore heuristic or evolutionary algorithms for near-optimal solution search [14], [56], [57]. Zhang et al. propose the traffic engineering framework TEDAT, which combines a greedy heuristic algorithm to realize dynamic resource allocation and backup route calculation [58]. Chen et al. propose an african vulture routing optimization (AVRO) algorithm to optimize routing in complex SDN environments [59]. *Despite these advancements, heuristic-based approaches fail to achieve accurate analysis of network states and timely management, which may limit their adaptability and efficiency in dynamic network.*

Recent studies aim to apply prediction to resource allocation [60], [61], [62], [63]. Liu et al. leverage ResNet-based link utilization prediction to refine path selection, mitigating scheduling inefficiencies caused by delayed traffic information [64]. Diel et al. propose an RL and SDN-aided congestion control tool, which can find better TCP parameters and achieve congestion prediction [65], [66]. In [67], a prediction-based DRL algorithm is proposed for adaptive scheduling in time-sensitive networks, using network prediction to accelerate training and improve convergence. Ref. [68] proposes an innovative federated imitation learning framework and a distributed scheduling algorithm based on traffic prediction to address resource allocation challenges in 6G networks. Ref. [69] proposes a downlink scheduling scheme based on DRL, integrating a prediction algorithm utilizing exponential smoothing recurrent neural networks within the controller. Furthermore, Kumari et al. propose a load balancing method based on load prediction to address controller load imbalances in SDN [70]. These works demonstrate that the combination of prediction and scheduling can enhance transmission mechanism [71], [72]. However, these studies usually focus on aspects such as task scheduling in specific scenarios, rather than providing overall transmission optimization or congestion avoidance.

In summary, traditional approaches are simple and easy to deploy, but their reactive nature constrains precise congestion management. While learning-based approaches achieve high performance, their lack of interpretability prevents them from guiding subsequent optimization. Policy mapping approaches offer a balanced solution, though their effectiveness diminishes with unpredictable traffic. Finally, heuristic-based approaches are low-complexity, but often at the expense of analytical accuracy and adaptability. Current transmission solutions rarely achieve a comprehensive balance among deployability, explainability, and adaptability within complex cross-domain network environments. In addition, although substantial progress has been made in traffic prediction, notable gaps remain in integrating traffic prediction and traffic management into a unified, adaptive framework. To address these limitations, it is essential to develop a closed-loop adaptive framework that enables continuous optimization throughout the network lifecycle. Such a framework should integrate precise sensing and analytical capabilities to provide reliable inputs for transmission decisions [73]. Additionally, it should be explainable to support efficient deployment and sustained improvement in highly dynamic environments.

TABLE III
THE DESCRIPTION OF THE MAIN NOTATIONS

Notation	Description
$\mathcal{G} =< \mathcal{N}, \mathcal{E} >$	$\mathcal{G}$ is the undirected network model graph. $\mathcal{N}$ is the set of nodes. $\mathcal{E}$ is the set of all links.
$\mathcal{B}$	The set of link capacities.
$\mathcal{T}$	The sequence of time slots.
$\mathcal{S}(t)$	The network flow set in slot t.
$\mathcal{F}(t)$	The flow demand set in slot t.
$\mathcal{P}_{ij}(t)$	The link sequence from node i to node j in slot t.
$\mathbf{A}$	The adjacency matrix.
$\mathbf{H}(t)$	The next-hop forwarding matrix in slot t.
$\mathbf{R}(t)$	The flow-link matrix in slot t.
$\mathbf{L}(t)$	The traffic distribution matrix in slot t.
$s_{ij}(t)$	The flow from node i to node j in slot t.
$f_s(t)$	The flow demand of flow s in slot t.
$p_{ij}^{(z)}(t)$	Recursive function to construct the path from node i to node j in slot t.
$h_{ij}(t)$	The next-hop forwarding node from node i toward node j in slot t.
$r_{se}(t)$	Binary indicator that denotes whether flow s passes through link e in slot t.
$l_e(t)$	The total traffic volume of all flows passing through link e in slot t.
$LI(t)$	The load-balancing indicator in slot t.
$u_e(t)$	The link utilization of link e in slot t.
a_{ij}	Binary indicator that denotes whether a link exists between node i and node j.
e_{ij}	The link between node i and node j.
b_e	The bandwidth of link e.
N	The total number of network nodes.

III. System Model and Problem Formulation

This paper considers a DCI scenario involving multiple DCs, as illustrated in Fig. 1. Multiple data flows are transmitted simultaneously, competing for network resources. This section models and analyzes the network transmission state in this scenario. The main parameters used in this paper are listed in Table III.

A. Network Model

We define $\mathcal{G} = \langle \mathcal{N}, \mathcal{E} \rangle$ as a fixed, connected, undirected graph representing the network topology. $\mathcal{N} = \{1, 2, \ldots, N\}$ is the set of all nodes, where N is the total number of network nodes. $\mathcal{E} = \{e_{ij} \mid \forall i, j \in \mathcal{N}, i \neq j\}$ is the set of all links, where e_{ij}

Authorized licensed use limited to: Universitas Indonesia. Downloaded on October 03,2026 at 15:35:47 UTC from IEEE Xplore. Restrictions apply.

represents the link between node i and node j. $\mathcal{B} = \{b_e \mid \forall e \in \mathcal{E}\}$ is the set of link capacities representing the bandwidth of each link in $\mathcal{E}$, where b_e denotes the bandwidth of the link e.

To describe the network transmission model clearly, we divide the transmission process into a sequence of time slots $\mathcal{T} = (1, 2, \ldots, T)$. As mentioned earlier, the network contains multiple pairs of communication flows. We assume that, in each time slot t ($t \in \mathcal{T}$), there is at most one data flow between any given terminal pair. Unlike conventional networks, where flows are typically identified by a five-tuple (source IP, destination IP, source port, destination port, and protocol), here a flow is defined solely by the terminal pair, i.e., all traffic from one node to another is regarded as a single flow. The data flows in the network can be represented by the set $\mathcal{S}(t) = \{s_{ij}(t) \mid \forall i, j \in \mathcal{N}, i \neq j\}$, where $s_{ij}(t)$ denotes the data flow from source node i to destination node j in slot t. Moreover, the demand for each flow is indicated by the flow demand set $\mathcal{F}(t) = \{f_s(t) \mid \forall s \in \mathcal{S}(t)\}$, where $f_s(t)$ is the demand of flow s in slot t.

To quantify the flow distribution, we introduce the adjacency matrix $\mathbf{A} = [a_{ij} \mid \forall i, j \in \mathcal{N}]$, where a_{ij} indicates whether a link exists between node i and node j:

$$a_{ij} = \begin{cases} 1, & \text{if } i \neq j \text{ and } e_{ij} \in \mathcal{E}, \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

Furthermore, we introduce the next-hop forwarding matrix:

$$\mathbf{H}(t) = \begin{bmatrix} h_{11}(t) & \cdots & h_{1N}(t) \\ \vdots & \ddots & \vdots \\ h_{N1}(t) & \cdots & h_{NN}(t) \end{bmatrix},$$

$$h_{ij}(t) \in \{0, 1, 2, \ldots, N\}, \ \forall i, j \in \mathcal{N}, \tag{2}$$

where $h_{ij}(t)$ denotes the next-hop forwarding node from node i toward node j in slot t. The value of $h_{ij}(t)$ is determined based on the following rules:

$$h_{ij}(t) = \begin{cases} 0, & \text{if } i = j, \\ j, & \text{if } i \neq j \text{ and } a_{ij} = 1, \\ x \in \mathcal{N} \setminus \{i, j\}, & \text{otherwise.} \end{cases} \tag{3}$$

$h_{ij}(t) = 0$ indicates that nodes i and j are the same node, while $h_{ij}(t) = j$ indicates that i is directly connected to j. Other situations indicate that communication between i and j requires forwarding by other nodes. In particular, $\mathbf{H}(t)$ must satisfy the topological and the path reachability constraints. The topological constraint requires that $h_{ij}(t)$ must be a neighboring node of i:

$$a_{ih_{ij}(t)} = 1, \quad i \neq j, \forall i, j \in \mathcal{N}, h_{ij}(t) \in \mathbf{H}(t). \tag{4}$$

The path reachability constraint requires that there be no loop in $\mathbf{H}(t)$, meaning that node i can communicate with node j within a bounded number of hops. Therefore, we define a recursive function $p_{ij}^{(z)}(t)$ to construct the path from node i to node j in slot t:

$$p_{ij}^{(0)}(t) = i, p_{ij}^{(1)}(t) = h_{ij}(t), \ldots,$$
$$p_{ij}^{(z)}(t) = h_{p_{ij}^{(z-1)}(t)j}(t), \ldots, p_{ij}^{(n)}(t) = j, \tag{5}$$

where

$$0 < z \leq n < N, \forall z \neq c, p_{ij}^{(z)}(t) \neq p_{ij}^{(c)}(t),$$
$$\forall i, j \in \mathcal{N}, i \neq j, h_{p_{ij}^{(z)}(t)j}(t) \in \mathbf{H}(t).$$

$n < N$ means that node i can reach node j in fewer than N hops (the maximum number of hops is $N - 1$, meaning that node i would traverse all other nodes to reach node j). $p_{ij}^{(z)}(t) \neq p_{ij}^{(m)}(t)$ is the explicit acyclic constraint, meaning that no node is visited more than once in the path. Therefore, the path from node i to node j in slot t can be represented by a link sequence:

$$\mathcal{P}_{ij}(t) = (e_{p_{ij}^{(0)}(t)p_{ij}^{(1)}(t)}, e_{p_{ij}^{(1)}(t)p_{ij}^{(2)}(t)}, \ldots, e_{p_{ij}^{(n-1)}(t)p_{ij}^{(n)}(t)}),$$
$$i \neq j, \forall i, j \in \mathcal{N}. \tag{6}$$

Although $\mathcal{P}_{ij}(t)$ is defined as a sequence generated by $p_{ij}^{(z)}(t)$, each element of $\mathcal{P}_{ij}(t)$ is recursively derived from $\mathbf{H}(t)$. Therefore, $\mathcal{P}_{ij}(t)$ is inherently constrained by $\mathbf{H}(t)$.

Furthermore, we define the flow-link matrix $\mathbf{R}(t) = [r_{se}(t) \mid \forall s \in \mathcal{S}(t), e \in \mathcal{E}]$, where $r_{se}(t)$ indicates whether flow s passes through link e in slot t:

$$r_{se}(t) = \begin{cases} 1, & \text{if } e \in \mathcal{P}_{ij}(t), \\ 0, & \text{otherwise,} \end{cases} \tag{7}$$

where i is the source node of flow s_{ij}, j is the destination node of flow s. Based on $\mathcal{F}(t)$ and $\mathbf{R}(t)$, we can derive the traffic distribution matrix $\mathbf{L}(t) = [l_e(t) \mid \forall e \in \mathcal{E}]$, where $l_e(t)$ denotes the total traffic volume of all flows passing through link e in slot t. $l_e(t)$ can be derived from the following equation:

$$l_e(t) = \sum_{s \in \mathcal{S}(t)} f_s(t) \cdot r_{se}(t), f_s(t) \in \mathcal{F}(t), r_{se}(t) \in \mathbf{R}(t). \tag{8}$$

Based on (8), the variation of $l_e(t)$ is influenced by $\mathbf{R}(t)$, which is indirectly constrained by the forwarding matrix $\mathbf{H}(t)$. As a result, $\mathbf{H}(t)$ plays a pivotal role in determining traffic distribution, since its update directly affects the traffic transmission path.

B. Problem Formulation

In DCI, uneven traffic distribution often leads to congestion on certain links while leaving others underutilized, degrading overall network performance, as illustrated in Fig. 2. To avoid network congestion and ensure transmission quality, incorporating a load-balancing metric as the optimization objective is necessary and meaningful. By minimizing traffic concentration and promoting the equitable utilization of network resources, this metric helps prevent localized congestion and enhances the network's robustness. Therefore, we adopt the load-balancing indicator proposed in [74] as our optimization objective. The load-balancing indicator is defined as the difference between the network's maximum and minimum link utilization, which can be expressed as:

$$LI(t) = \max_{e \in \mathcal{E}}(u_e(t)) - \min_{e \in \mathcal{E}}(u_e(t)), \tag{9}$$

where $LI(t)$ denotes the load-balancing indicator in slot t, and $u_e(t)$ denotes the utilization of link e in slot t. The utilization

Authorized licensed use limited to: Universitas Indonesia. Downloaded on October 03,2026 at 15:35:47 UTC from IEEE Xplore. Restrictions apply.

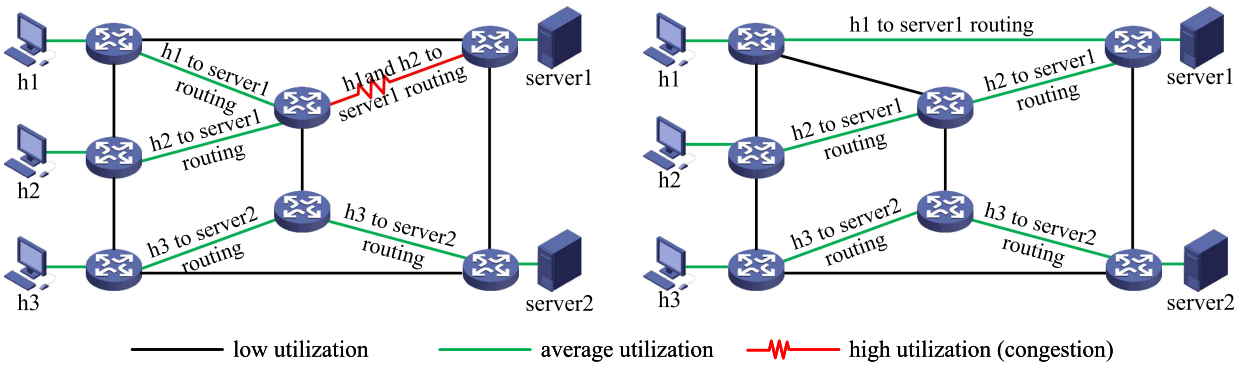


Fig. 2. The relationship between data flow distribution and network congestion in multi-flow transmission scenarios.

$u_e(t)$ can be expressed as:

$$u_e(t) = \frac{l_e(t)}{b_e}, \quad \forall e \in \mathcal{E}. \tag{10}$$

A larger $LI(t)$ indicates that most flows tend to be concentrated on certain links, increasing the risk of congestion. Conversely, a smaller $LI(t)$ reflects a more balanced distribution of flows, reducing the likelihood of congestion. Therefore, we formulate a traffic scheduling optimization problem to avoid congestion:

$$\min_{\mathbf{H}(t)} LI(t), \tag{11}$$

$$\text{s.t.} \sum_{s \in \mathcal{S}(t)} f_s(t) \cdot r_{se}(t) \leq b_e, \quad \forall e \in \mathcal{E}, t \in \mathcal{T},$$

$$(4) \text{ and } (5). \tag{12}$$

Specifically, we formulate the problem as finding a forwarding strategy $\mathbf{H}(t)$ that minimizes the load-balancing indicator $LI(t)$ in slot t, subject to topological and path reachability constraints. Constraint (12) ensures no link exceeds its bandwidth capacity.

By reducing the objective of (11) to a feasibility verification problem (i.e., determining whether any valid $\mathbf{H}(t)$ exists given constraints), the problem becomes equivalent to the integer multi-commodity flow problem, which is known to be NP-hard [75], [76]. Consequently, (11) is NP-hard, and the number of feasible combinations of $\mathbf{H}(t)$ grows exponentially with network scale and time granularity. This exponential growth poses two critical challenges for congestion avoidance in dynamic networks. First, the vast solution space increases the time of finding the optimal solution, which poses substantial challenges to real-time scheduling. Second, DCI network traffic is highly dynamic, where any delay in computing and deploying strategies may lead to a mismatch between the strategy and the actual network state, thereby reducing the effectiveness of transmission [77]. Moreover, searching such a vast solution space demands substantial computational resources, while the processing capabilities of routers are limited, further hindering real-time responsiveness. *Therefore, it is crucial to design an efficient traffic management scheme that balances real-time responsiveness and computational complexity.* To this end, we propose a traffic management framework that integrates traffic prediction with traffic scheduling. The framework predicts the network state in the upcoming time slot and, based on the predicted traffic, generates fast and adaptive proactive scheduling strategies to achieve congestion avoidance.

IV. Framework Design and Algorithm Description

In this section, we propose PPCA for dynamic traffic management in DCI. We first introduce the core framework and workflow of PPCA, and then provide a detailed overview of the key algorithms.

A. Framework Design

As mentioned in Section III, in cross-domain, highly dynamic DCI scenarios, real-time traffic management imposes two critical requirements for strategy generation: timely responsiveness and sustained optimization. Traditional reactive or direct online optimization methods face two limitations: they typically update strategies only after congestion occurs, and they require significant computational resources from routers. To address these requirements, we design the core framework of PPCA. Inspired by DTN, the framework constructs a virtual mirror of the physical network, enabling dynamic closed-loop network control, which covers state collection, accurate prediction, strategy generation, and strategy deployment. As illustrated in Fig. 3, the framework comprises two planes: the data transmission plane with actual network components, and the network control plane with the network twin and congestion avoidance modules. The network control module acts as the hub connecting these two planes. The following introduces each module.

- *Network Control Module:* Serves as the state collection and strategy deployment interface for closed-loop network management. Specifically, the network control module connects to the switches through a controller. It periodically collects network state metrics (network traffic, anomalies, and device connectivity) to construct the network state set $\mathcal{V}(t)$ and provides $\mathcal{V}(t)$ to the network twin module. Meanwhile, the network control module obtains the transmission strategy $\tilde{\mathbf{H}}_\delta(t+1)$ from the congestion avoidance module and deploys it to the physical network, enabling efficient cross-domain traffic management without terminal involvement.
- *Network Twin Module:* Constructs a virtual mirror of the physical network. Leveraging machine learning, the

Authorized licensed use limited to: Universitas Indonesia. Downloaded on October 03,2026 at 15:35:47 UTC from IEEE Xplore. Restrictions apply.

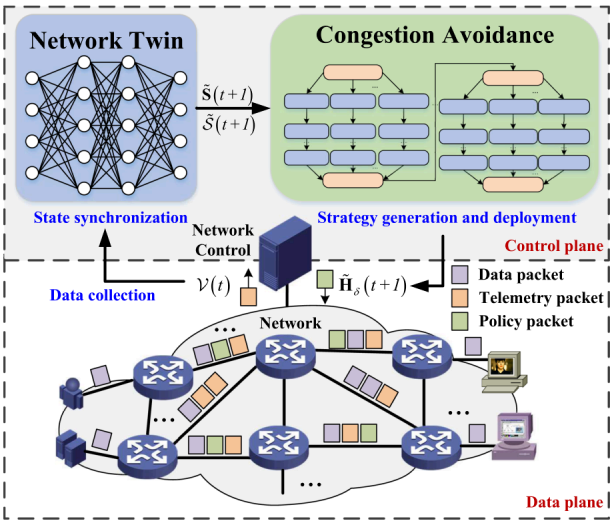


Fig. 3. The core framework of PPCA.

module captures traffic spatiotemporal dependencies and predicts future traffic distribution trends $\tilde{\mathbf{S}}(t+1)$. Subsequently, the predicted traffic information $\tilde{\mathcal{S}}(t+1)$ and $\tilde{\mathbf{S}}(t+1)$ are forwarded to the congestion avoidance module, serving as prior knowledge for strategy optimization.

- *Congestion Avoidance Module:* Generates a proactive scheduling strategy to prevent congestion. Taking the predicted traffic information as input, the congestion avoidance module constructs a scheduling decision model and emulates candidate strategies in the virtual environment. The module evaluates each strategy based on its load-balancing indicator, assigns a performance score, and adaptively selects the optimal strategy $\tilde{\mathbf{H}}_\delta(t+1)$. $\tilde{\mathbf{H}}_\delta(t+1)$ is then sent to the network control module for deployment, ensuring resource allocation adjustments before congestion occurs.

Unlike reactive approaches, the framework provides a global, and real-time view of cross-domain network states. By combining the virtual mirror with strategy emulation in the virtual environment, the framework can evaluate multiple strategies without burdening physical infrastructure, thereby eliminating risks and latency from direct physical deployment. In practice, the framework executes the following iterative cycle:

- *Data Collection:* The network control module connects to the switches and periodically collects network state information to form a global view for the upper-layer.
- *State Synchronization:* The updated network state sets are pushed to the network twin module, which performs data cleaning and data visualization, as well as network traffic analysis and prediction. The analysis results are then pushed to the congestion avoidance module.
- *Strategy Generation and Deployment:* The congestion avoidance module generates the optimal transmission strategy using predicted information and sends it to the control module. After receiving the strategy, the network control module deploys it to the switch, enabling adaptive network optimization without terminal involvement.

Notably, this process is not a one-time execution, but rather a continuous, iterative cycle. By continuously synchronizing physical network states to the virtual mirror, the framework dynamically optimizes and updates scheduling strategies, achieving both fast responsiveness to bursty traffic and global optimal resource allocation. The collaborative operation of all modules establishes a full-cycle proactive management paradigm, shifting network control from post-failure remediation to predictive optimization and continuous adaptation.

B. Neighbor-Assisted Traffic Prediction

In Section III, we analyze the accuracy and delay issues between the network traffic state analysis and the strategy generation. To address these issues, we propose leveraging traffic prediction to enable real-time scheduling, thereby improving both the proactivity and reliability of decision-making. Conventional network traffic prediction methods typically employ LSTM. However, the study in [78] demonstrates that the TCN architecture outperforms typical recurrent models such as LSTM across multiple tasks, exhibiting superior capability in capturing long-term dependencies. Motivated by these findings, we incorporate TCN in our traffic prediction method to enhance prediction accuracy. At the same time, it is important to note that network traffic exhibits both temporal and spatial correlations. Temporal correlation reflects the variation patterns of traffic parameters—such as rate and bandwidth utilization—over time. Correspondingly, the spatial correlation reflects the interaction characteristics of traffic between nodes and links in the topology structure. While TCNs are effective at capturing temporal dependencies, they inherently fail to capture explicit spatial correlations. To address this limitation, we propose a neighbor-assisted traffic prediction method. This method builds customized TCN prediction models for each switch and extracts neighbor information as model inputs, thereby enhancing their capability to capture long-term dependencies and local spatial features. Then, the proposed prediction method is embedded into the network twin module to generate traffic predictions at one-second intervals, which are delivered to the congestion avoidance module in real time. The workflow of the method is illustrated in Fig. 4, with details provided in Algorithm 1.

1) Data Preprocessing: The network information $\mathcal{V}(t)$ collected by the network control module comprises various data, including network connectivity information, which must be preprocessed to extract traffic-driven features and ensure compatibility with the prediction model. To this end, the method first converts $\mathcal{V}(t)$ into a switch-to-terminal traffic matrix, denoted as $\mathbf{S}(t)$, which represents the traffic volume between each switch and terminal in slot t. In addition, network traffic data often exhibits an uneven distribution, characterized by numerous extreme values (e.g., some values close to zero while others exceed 10,000). Such disparities can affect model convergence and prediction accuracy. To address this issue, the method applies

Authorized licensed use limited to: Universitas Indonesia. Downloaded on October 03,2026 at 15:35:47 UTC from IEEE Xplore. Restrictions apply.

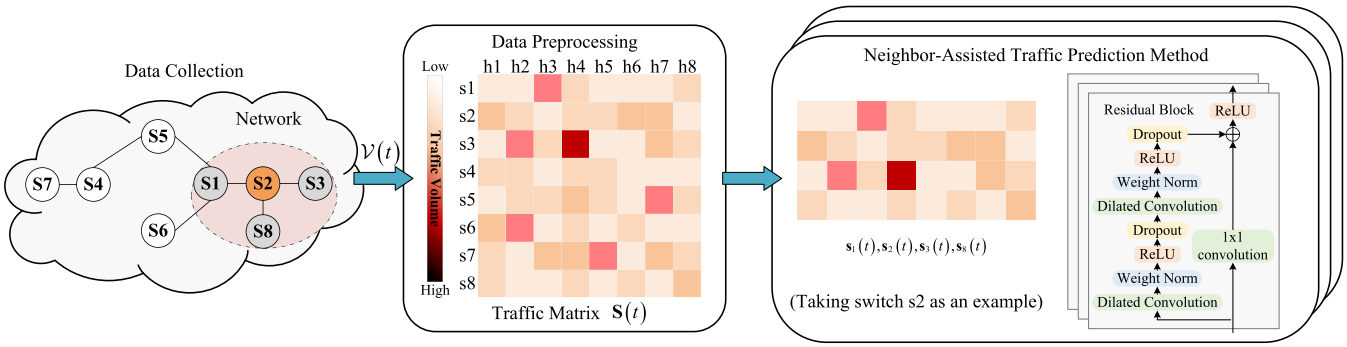


Fig. 4. The workflow of neighbor-assisted traffic prediction method.

Algorithm 1: Neighbor-Assisted Traffic Prediction Method.

Input: Network information $\mathcal{V}(t)$ in slot t;
Output: Predicted traffic matrix $\tilde{\mathbf{S}}(t+1)$ in slot $t+1$;
Get the network information $\mathcal{V}(t)$ from network control module;
Calculate the switch-to-terminal traffic matrix $\mathbf{S}(t)$ and $\mathcal{G} = \langle \mathcal{N}, \mathcal{E} \rangle$ based on $\mathcal{V}(t)$;
Normalize the input of the prediction model based on Eq. (13);
for *switch nodes* $i \in \mathcal{N}$ **do**
  Get the neighbor node $\{j, \ldots, o\}$ of i based on $\mathcal{V}$;
  Get neighbor nodes' traffic vector $\mathbf{s}_j(t), \ldots, \mathbf{s}_o(t)$ based on $\mathbf{S}(t)$;
  Generate prediction model TCN_i for i ;
  Calculate the predict vector of i in slot $t+1$ based on $\tilde{\mathbf{s}}_i(t+1) = TCN_i(\mathbf{s}_j(t), \ldots, \mathbf{s}_o(t), \mathbf{s}_i(t))$;
Return $\tilde{\mathbf{S}}(t+1), \tilde{\mathcal{S}}(t+1)$;

a combination of logarithmic transformation and min-max normalization for data preprocessing. The preprocessing equation is as follows:

$$\mathbf{S}(t) = \frac{\ln(\mathbf{S}'(t)) - \min(\ln(\mathbf{S}'(t)))}{\max(\ln(\mathbf{S}'(t))) - \min(\ln(\mathbf{S}'(t)))}, \tag{13}$$

where $\mathbf{S}'(t)$ denotes the traffic matrix in slot t before normalization, and $\mathbf{S}(t)$ denotes the traffic matrix in slot t after normalization.

In contrast to simple min-max normalization, which only performs linear scaling, logarithmic transformation compresses heavy-tailed distributions and reduces the impact of outliers, thereby making the data more suitable for model training. However, applying logarithmic transformation alone may still leave heterogeneous ranges, affecting convergence during training. By applying min-max normalization after logarithmic compression, the data are mapped to a uniform range, preserving relative differences while ensuring numerical stability. Alternative preprocessing methods, such as Z-score normalization or piecewise clipping, either fail to mitigate the influence of outliers or distort inherent traffic patterns, which could compromise prediction accuracy. In contrast, the combination of logarithmic transformation and min-max normalization balances these concerns, improving model stability, accelerating convergence, and enhancing prediction accuracy.

2) Neighbor-Assisted Traffic Prediction Method: After preprocessing, the data is fed into the prediction model. The model is built upon TCNs and consists of multiple stacked residual blocks to capture temporal dependencies in traffic data. Each residual block contains two temporal blocks, and each temporal block consists of the following components: a dilated causal convolution, weight normalization, a rectified linear unit (ReLU) activation function, and a dropout layer. The dilated causal convolution employs zero padding to preserve the input sequence length, followed by truncation of the rightmost padded values to ensure strict causality. By progressively increasing the dilation factors, the receptive field of the convolutional kernel is expanded, allowing the capture of long-term dependencies. After the convolution, weight normalization is applied to stabilize the training process by decoupling the direction and magnitude of the convolutional filters. Then, a ReLU activation function is introduced to enhance non-linearity and improve model expressiveness. Finally, a dropout layer is applied to regularize the network and reduce the risk of overfitting. In addition, each residual block includes a residual connection, which directly passes the input to the output, thereby facilitating gradient flow during backpropagation and enhancing the trainability of deep networks. When the feature dimensions of the input and output do not match, a 1×1 convolution is applied to align the dimensions, ensuring the element-wise addition in the residual connection can be performed correctly.

As the residual blocks are stacked, the receptive field of the TCN expands rapidly. Specifically, the size of the receptive field is jointly determined by the kernel size ks, the dilation factors d, and the number of residual blocks L. The incremental receptive field contributed by a single dilated convolution layer is $(ks-1) \cdot d$ [79]. Since each residual block performs two convolution operations, the contribution of one residual block to the receptive field is $2 \cdot (ks-1) \cdot d$. For a TCN with L residual blocks, the overall receptive field RF can be expressed as:

$$RF = 1 + \sum_{i=0}^{L-1} 2 \cdot (ks-1) \cdot d_i, \tag{14}$$

where d_i denotes the dilation factor of the i-th residual block. By incorporating the TCN architecture, the method enables

Authorized licensed use limited to: Universitas Indonesia. Downloaded on October 03,2026 at 15:35:47 UTC from IEEE Xplore. Restrictions apply.

parallel processing of traffic sequences. Moreover, by adjusting the number of layers, kernel size, and dilation factors, the method flexibly adjusts its receptive field, thereby capturing long-term temporal dependencies.

While conventional TCNs are effective in capturing long-term temporal dependencies of traffic, they exhibit three critical limitations in large-scale network prediction. First, TCNs lack the ability to capture explicit spatial correlations, which are essential for reflecting the interactions among nodes and links in the network topology. Second, TCNs need to take in the raw sequence up to the effective history length, leading to more memory overhead. Third, when scenario changes, performance may degrade due to mismatched receptive fields. To address these limitations, the method introduces a neighbor-assisted prediction design. Instead of constructing a single global TCN, the method establishes a customized traffic prediction model for each switch, meaning that each model predicts the traffic matrix from a single switch to all terminals. This design mitigates the risk of receptive field mismatch due to topology changes and captures the individual traffic patterns and local dynamics of each switch, avoiding the averaging effect inherent in global models. Furthermore, for each switch-specific model, we extract network connectivity information from $\mathcal{V}(t)$ and limit the input data to the data from the switch itself and its neighboring nodes. This design not only reduces the dimensionality of the input space but also emphasizes local spatial correlations, enhancing the model's sensitivity to local traffic variations and improving prediction accuracy.

Assuming that the network contains y switches, there are a total of y models. Taking switch $s2$ as an example, its neighboring nodes consist of switches $s1$, $s3$, and $s8$. Accordingly, the prediction model for $s2$ uses only the traffic data from $s1$, $s2$, $s3$, and $s8$ as input:

$$\tilde{\mathbf{s}}_2(t+1) = TCN_2(\mathbf{s}_1(t), \mathbf{s}_2(t), \mathbf{s}_3(t), \mathbf{s}_8(t)),$$
$$\mathbf{s}_1(t), \mathbf{s}_2(t), \mathbf{s}_3(t), \mathbf{s}_8(t) \in \mathbf{S}(t), \tag{15}$$

where $\mathbf{s}_1(t)$, $\mathbf{s}_2(t)$, $\mathbf{s}_3(t)$, and $\mathbf{s}_8(t)$ denote the traffic vector of switches $s1$, $s2$, $s3$, and $s8$ in slot t, respectively. $\tilde{\mathbf{s}}_2(t+1)$ denotes the predicted traffic vector for switch $s2$ in slot $t+1$, and $TCN_2()$ denotes the prediction model of switch $s2$. In this paper, a tilde over a variable (e.g., $\tilde{x}$) indicates its predicted value.

By aggregating the outputs of all switch-specific models, the method reconstructs the predicted switch-to-terminal traffic matrix $\tilde{\mathbf{S}}(t+1)$ and generates the corresponding predicted flow information $\tilde{\mathcal{S}}(t+1)$ in slot $t+1$.

C. Exploration-Exploitation-Based Flow Scheduling

As analyzed in Section III, real-time traffic scheduling in inter-domain dynamic networks must satisfy two requirements: (i) fast search in the feasible space for real-time decisions, and (ii) sufficient interpretability for subsequent network management. To address these challenges, we propose an exploration-exploitation-based flow scheduling algorithm, which is embedded in the congestion avoidance module to achieve efficient and interpretable scheduling. Based on the definition of $\mathbf{H}(t)$, any change in $\mathbf{H}(t)$ alters the transmission path of a flow, thereby affecting traffic distribution. Therefore, the algorithm regards the change of one value in $\mathbf{H}(t)$ as an action, and the subset of actions is regarded as a strategy. The algorithm then integrates an exploration–exploitation mechanism that supports parallel strategy evaluation and adaptive expansion of action combinations, thereby enhancing computational efficiency. Specifically, the mechanism maintains a probability matrix $\mathbf{pro}$ for action selection and a score matrix $\mathbf{sco}$ for performance evaluation. Based on $\mathbf{pro}$, the mechanism dynamically selects actions and generates candidate strategies in parallel to ensure efficient exploration of diverse action combinations. For each candidate strategy, the load-balancing indicator (LI) is calculated based on the predicted traffic and used as the evaluation criterion. The strategy with the minimum LI is designated as the optimal strategy δ. Then the mechanism updates $\mathbf{sco}$ and $\mathbf{pro}$ based on δ, increasing the selection probability of favorable actions. Through iterative updates, the mechanism converges rapidly to a globally near-optimal policy while maintaining manageable computational overhead. Unlike RL, the proposed algorithm does not rely on complex reward functions, long-term policy learning, or explicit state transition modeling. It directly evaluates candidate strategies using a scoring metric derived from load-balancing indicators, enabling immediate and interpretable decisions. The algorithm iteratively updates action probabilities through the exploration–exploitation mechanism, which allows efficient search and faster convergence to the optimal strategy, rather than learning a policy function over the environment. By incorporating predicted traffic, the algorithm can directly support real-time congestion avoidance and facilitate more effective network management and optimization. The details of the algorithm are described below, as illustrated in Fig. 5 and Algorithm 2.

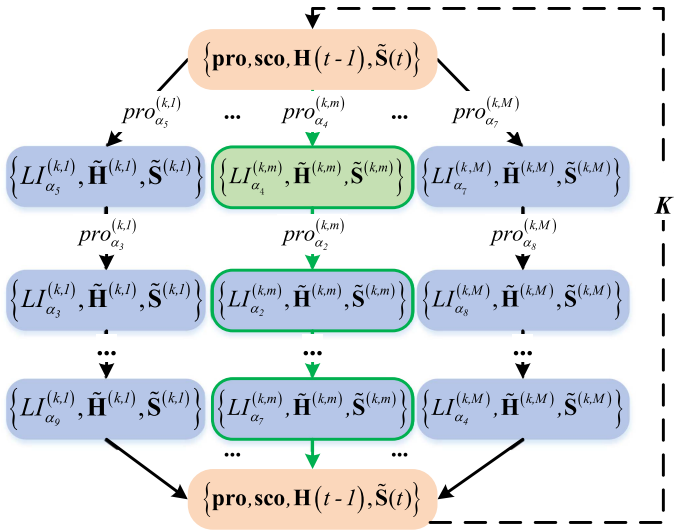


Fig. 5. The design of the flow scheduling algorithm.

1) Algorithm Initialization: In the algorithm, a proactive scheduling action is defined as:

$$\alpha = (\tilde{f}_s^i(t), j), \tilde{f}_s^i(t) > 0, e_{ij} \in \mathcal{E}, s \in \tilde{\mathcal{S}}(t). \tag{16}$$

Equation (16) indicates that for a predicted flow s at node i, the next-hop forwarding node is j. For action α, only existing flows can be scheduled, which means $\tilde{f}_s^i(t) > 0$ and $s \in \tilde{\mathcal{S}}(t)$.

Authorized licensed use limited to: Universitas Indonesia. Downloaded on October 03,2026 at 15:35:47 UTC from IEEE Xplore. Restrictions apply.

Moreover, the selected next-hop forwarding node must comply with network connectivity, that is, $e_{ij} \in \mathcal{E}$. Assuming node i has three flows and two connected nodes. In this case, each flow has two possible next-hop forwarding choices, resulting in six actions at node i. Based on the predicted traffic information provided by the network twin module, the algorithm constructs the action set $\mathcal{A}(t)$. For action α, pro_α denotes its selection probability, and sco_α represents its load-balancing score.

At initialization, $\mathbf{pro}$ is set to a uniform distribution, where all elements are equal and sum to 1. Meanwhile, $\mathbf{sco}$ is initialized with equal values, ensuring all actions start with the same probability and score (*Line 1*). The algorithm takes $\mathcal{A}(t)$, $\tilde{\mathbf{S}}(t)$ and the current next-hop forwarding matrix $\mathbf{H}(t-1)$ as input, and iteratively updates the optimal strategy δ under the exploration-exploitation mechanism, finally outputs the optimal forwarding matrix $\tilde{\mathbf{H}}_\delta(t)$ for the predicted traffic. Here, δ is defined by $\tilde{\mathbf{H}}_\delta(t)$ along with its associated minimum load-balancing indicator LI_δ and the resulting traffic distribution matrix $\tilde{\mathbf{S}}_\delta(t)$. Initially, δ is initialized as $\{LI_\delta, \tilde{\mathbf{H}}_\delta(t), \tilde{\mathbf{S}}_\delta(t)\} = \{LI, \mathbf{H}(t-1), \tilde{\mathbf{S}}(t)\}$ (*Line 3*). LI_δ denotes the initial load-balancing indicator, calculated based on $\mathbf{H}(t-1)$ and $\tilde{\mathbf{S}}(t)$ (*Line 2*). Consequently, the selection probabilities of the actions in $\mathbf{H}(t-1)$ are set to zero to avoid re-selection in subsequent strategy construction, thereby reducing computational complexity (*Line 4*). The initial average transmission hop matrix $\mathbf{hop}$ corresponding to $\mathbf{H}(t-1)$ is also computed, which serves as a condition for the subsequent updates of $\mathbf{pro}$ (*Line 2*). Each element hop_{ij} in $\mathbf{hop}$ represents the transmission hop count from source node i to destination node j. For strategies with higher transmission hops, their corresponding scores are assigned lower values in subsequent updates.

2) Exploration-Exploitation Mechanism: We divide the exploration-exploitation mechanism into two parts: the exploration mechanism for parallel strategy assessment and the exploitation mechanism for probability update. Specifically, the mechanism includes K iterations (*Line 5*). In each iteration, the exploration mechanism constructs M candidate strategies in parallel based on $\mathbf{pro}$, evaluates them to identify the optimal strategy, and updates $\mathbf{sco}$ accordingly (*Lines 6, 7*). After the exploration mechanism, the exploitation mechanism updates $\mathbf{pro}$ based on $\mathbf{sco}$ and uses updated $\mathbf{pro}$ as input for the next iteration of the exploration mechanism.

We first describe the exploration mechanism. At the beginning of each iteration, the parallel construction shares the same initial conditions $\{\mathbf{pro}, \mathbf{sco}, \mathbf{H}(t-1), \tilde{\mathbf{S}}(t)\}$ (*Line 8*). The following is the construction process of a candidate strategy in the exploration mechanism.

- *Action selection and validation (Lines 10-12):* For strategy m_k, the mechanism first selects an action α based on $\mathbf{pro}^{(k,m)}$, where k denotes the iteration index and m denotes the index of the candidate strategy constructed in parallel. If α leads to a loop in $\tilde{\mathbf{H}}^{(k,m)}$ or exceeds the link capacity, the mechanism returns to the action selection process.
- *Strategy updating (Lines 13-15):* If α satisfies the bandwidth and loop constraints, the mechanism calculates $LI_\alpha^{(k,m)}$ and updates $\tilde{\mathbf{H}}^{(k,m)}$ and $\tilde{\mathbf{S}}^{(k,m)}$. After that, $\{LI_\alpha^{(k,m)}, \tilde{\mathbf{H}}^{(k,m)}, \tilde{\mathbf{S}}^{(k,m)}\}$ is then added to strategy m_k, as shown by the green box of Fig. 5. Furthermore, the selection probability of α in $\mathbf{pro}^{(k,m)}$ is set to zero to avoid reselection within m_k, thereby reducing computational complexity.
- *Optimal strategy update (Lines 16-18):* After updating the strategy, the mechanism performs optimal strategy evaluation. If $LI_\alpha^{(k,m)}$ is lower than LI_δ, δ will be updated:

$$\{LI_\delta, \tilde{\mathbf{H}}_\delta(t), \tilde{\mathbf{S}}_\delta(t)\} = \{LI_\alpha^{(k,m)}, \tilde{\mathbf{H}}^{(k,m)}, \tilde{\mathbf{S}}^{(k,m)}\}. \tag{17}$$

Then, the score of α is updated as follows:

$$sco_\alpha = \rho \cdot sco'_\alpha + Q \cdot (1 - LI_\alpha^{(k,m)}), \tag{18}$$

where sco'_α denotes the score of α before the update, ρ is the score decay factor, and Q is the score increment factor. The first part of (18), $\rho \cdot sco'_\alpha$, represents the weighted contribution of the previous score. The second part of (18), $Q \cdot (1 - LI_\alpha^{(k,m)})$, represents the incremental contributed by the current update. It is worth noting that to reduce computational complexity, the mechanism updates the score only for the currently selected action. Additionally, $\mathbf{hop}$ is updated according to (19) for subsequent updates of $\mathbf{pro}$:

$$\mathbf{hop} = ((u-1)\mathbf{hop}' + \mathbf{hop}_\alpha)/u, \tag{19}$$

where $\mathbf{hop}'$ denotes the average transmission hop matrix before the update, $\mathbf{hop}_\alpha$ denotes the transmission hop matrix of α, and u denotes the number of updates for δ.

- *Repeat action selection (Lines 9, 19):* If δ is not updated, the mechanism repeatedly selects actions based on $\mathbf{pro}^{(k,m)}$ until all actions are chosen.

After the above process, a strategy is formed with $\{\mathbf{pro}, \mathbf{sco}, \mathbf{H}(t-1), \tilde{\mathbf{S}}(t)\}$ serving as the root node, as indicated by the green line of Fig. 5. After constructing M strategies in parallel, the exploration mechanism is completed.

After the exploration mechanism, the exploitation mechanism updates $\mathbf{pro}$ based on $\mathbf{sco}$ (*Lines 20, 21*). Specifically, the mechanism first calculates the weight of each action:

$$\omega_\alpha = (sco_\alpha)^\varphi \left(\frac{1}{hop_s}\right)^\phi,$$
$$\forall \alpha \in \mathcal{A}(t), s \in \tilde{\mathcal{S}}(t), sco_\alpha \in \mathbf{sco}, hop_s \in \mathbf{hop}, \tag{20}$$

where hop_s represents the transmission hop from the source node to the destination node of flow s, ω_α represents the unnormalized weight of action α, φ refers to the importance parameter for the score, and ϕ refers to the importance parameter for the transmission hop. The mechanism then normalizes the weights to obtain $\mathbf{pro}$:

$$pro_\alpha = \frac{\omega_\alpha}{W}, \forall \alpha \in \mathcal{A}(t), pro_\alpha \in \mathbf{pro}, W = \sum_{\alpha \in \mathcal{A}(t)} \omega_\alpha. \tag{21}$$

Equation (21) ensures that the elements of $\mathbf{pro}$ sum to 1. Actions with higher transmission hops have lower selection

Authorized licensed use limited to: Universitas Indonesia. Downloaded on October 03,2026 at 15:35:47 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: Exploration-Exploitation-Based Flow Scheduling Algorithm.

Input: Action set $\mathcal{A}(t)$, predicted traffic information $\tilde{\mathbf{S}}(t)$, current forwarding matrix $\mathbf{H}(t-1)$;
Output: Forwarding matrix $\tilde{\mathbf{H}}_\delta(t)$ for $\tilde{\mathbf{S}}(t)$;
1 Initialize **pro** and **sco**;
2 Initialize LI and **hop** based on $\mathbf{H}(t-1)$ and $\tilde{\mathbf{S}}(t)$;
3 Set $u=0$ and $\{LI_\delta, \tilde{\mathbf{H}}_\delta(t), \tilde{\mathbf{S}}_\delta(t)\} = \{LI, \mathbf{H}(t-1), \tilde{\mathbf{S}}(t)\}$;
4 Set the probabilities of actions in $\mathbf{H}(t-1)$ to 0;
5 **for** $k = 1, \ldots, K$ **do**
6 **Parallel Execution:** Spawn M parallel workers;
7 **for** $m = 1, \ldots, M$ **do in parallel**
8 Set initial conditions with $\mathbf{pro}^{(k,m)} = \mathbf{pro}, \tilde{\mathbf{H}}^{(k,m)} = \mathbf{H}(t-1), \tilde{\mathbf{S}}^{(k,m)} = \tilde{\mathbf{S}}(t)$;
9 **while** $\mathbf{pro}^{(k,m)} = 0$ **do**
10 Select action α from $\mathcal{A}(t)$ based on $\mathbf{pro}^{(k,m)}$;
11 **if** *loop_detection() or bandwidth_limitation() is false* **then**
12 Continue;
13 Update $\tilde{\mathbf{H}}^{(k,m)}, \tilde{\mathbf{S}}^{(k,m)}$ based on α;
14 Calculate $LI_\alpha^{(k,m)}$ based on $\tilde{\mathbf{H}}^{(k,m)}$ and $\tilde{\mathbf{S}}^{(k,m)}$;
15 Set $pro_\alpha^{(k,m)} = 0$ and add $\{LI_\alpha^{(k,m)}, \tilde{\mathbf{H}}^{(k,m)}, \tilde{\mathbf{S}}^{(k,m)}\}$ to strategy m_k;
16 **if** $LI_\alpha^{(k,m)} \le LI_\delta$ **then**
17 $u = u + 1$;
18 Update δ, **sco** and **hop** based on Eq. (17), Eq. (18) and Eq. (19);
19 Break;
20 **Synchronize:** All parallel workers finished;
21 Update **pro** based on Eq. (20) and Eq. (21);
22 Return $\tilde{\mathbf{H}}_\delta(t)$;

probabilities, while actions with higher scores have higher probabilities. Once **pro** is updated, it is fed back as a condition into the exploration mechanism (*Lines 5, 8*).

After K iterations, the algorithm final outputs $\tilde{\mathbf{H}}_\delta(t)$ with the lowest load-balancing indicator, guiding the scheduling of predicted traffic and achieving congestion avoidance (*Line 22*). By dynamically adjusting **pro** and **sco**, the algorithm maintains sufficient exploration in the early stages to avoid local optima, while gradually increasing the selection probability of the optimal strategy as iterations proceed. This iterative process enables adaptive adjustment, improving robustness to environmental changes and avoiding the performance loss caused by fixed strategies.

V. Simulation Results and Analysis

To further evaluate the performance of PPCA, this section develops a prototype system based on programmable data planes (PDP) and compares its performance.

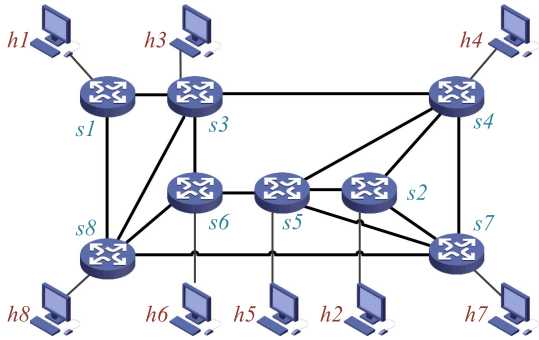


Fig. 6. Simulation topology.

TABLE IV
Hyperparameter Configuration for the Prediction Model

Parameter	Value	Parameter	Value
Learning rate	0.002	Sequence length	31
Dropout rate	0.1	Training data rate	0.95
Weight decay	5e−8	Batch size	32
Epochs	20	Optimizer	Adam

A. Simulation Setup

1) Simulation Configuration: We use the same topology as [80], as shown in Fig. 6. This topology has been widely used in research on network routing, scheduling, and congestion control. Furthermore, we select real-world traffic data from the MAWI Working Group as the input traffic for the topology [81]. The dataset is divided into eight equal parts, each of which is replayed on a separate terminal using the tcpreplay tool. The simulation is implemented on laboratory servers using PDP and Mininet, and is conducted on a Suma X785-H30 server equipped with a Hygon 7151 CPU at 2.0 GHz, 256 GB of RAM, 3.6 TB of storage, running Ubuntu 20.04.1 LTS.

The hyperparameter settings for the prediction model are presented in Table IV. It is worth noting that, in a recent study on cross-datacenter traffic analysis, the authors observed minute-level variation patterns in inter-datacenter traffic [82]. In their work, link utilization was sampled every 30 seconds. Therefore, this paper adopts an input window of 31 seconds to capture the primary dynamics of inter-datacenter traffic.

To determine the parameter settings of the proposed scheduling algorithm, we evaluate its performance under various values of parallel workers (M) and iterations (K). Since the paper is based on simulation experiments, computational overhead and memory usage cannot serve as valid metrics for evaluating the algorithm's performance. Specifically, we measured the execution time, the MLU reduction ratio, the number of optimal strategy updates, and the update gain. Execution time reflects resource consumption, as longer runtimes imply higher usage. The MLU reduction ratio serves as a performance metric, where a larger value indicates better performance. The number of updates characterizes the algorithm's exploration capability, with higher values denoting stronger exploration. The update gain, defined as the ratio between the MLU reduction ratio and the number of optimal strategy updates, represents the average MLU reduction per update and indicates the convergence progress. A decreasing

Authorized licensed use limited to: Universitas Indonesia. Downloaded on October 03,2026 at 15:35:47 UTC from IEEE Xplore. Restrictions apply.

TABLE V
ALGORITHM PERFORMANCE UNDER DIFFERENT VALUES OF M AND K

	Execution time (ms)	Reduction ratio (%)	Update times	Update gain
$K = 10, M = 10$	272.65	22.28	31	0.719
$K = 10, M = 15$	351.82	25.39	48	0.529
$K = 15, M = 15$	483.72	26.03	57	0.457
$K = 20, M = 15$	706.69	26.48	115	0.230
$K = 25, M = 15$	869.08	27.16	169	0.161
$K = 25, M = 20$	906.78	26.54	150	0.177

update gain reflects diminishing returns, implying that the algorithm is approaching convergence. The simulation results are shown in Table V. It can be observed that as K and M increase, the update gain decreases. This indicates that further updates do not improve the MLU reduction ratio, revealing a convergence threshold of the algorithm. Meanwhile, the execution time also increases, implying higher resource consumption. For larger K and M, the computation time exceeds 1 s, failing to meet the second-level scheduling requirement of PPCA. Furthermore, when M is set to 15, MLU consistently converges to over 26% for $K > 15$. When $K = 25$, the update gain becomes stable, indicating that the algorithm reaches a relatively steady state. Therefore, the algorithm parameters are set to $K = 25$ and $M = 15$ in this study.

2) Comparison Algorithms: We evaluate PPCA from two perspectives: prediction accuracy and transmission performance. For prediction accuracy, we compare PPCA with several widely used prediction models, including a baseline TCN (a single TCN model performing traffic prediction for the entire network), LSTM, CNN, GCN, as well as their combinations and variants TCN+LSTM, CCN+LSTM, LSTM+GCN, AttCNN [83], and ResLSTM [84]. The baseline TCN is included to illustrate the differences between a conventional TCN and PPCA's neighbor-aware traffic prediction method. For transmission performance, we compare five algorithms: shortest path (SP), non-predictive traffic scheduling (NP), random scheduling (RS), greedy scheduling (GS), and AttCNN-PCA [83]. SP does not consider network conditions, choosing the transmission link with the fewest hops. Typical routing protocols, such as open shortest path first (OSPF) and routing information protocol (RIP), utilize SP for routing. NP uses measured rather than predicted traffic as input to the congestion avoidance module. By comparing PPCA with NP, we can observe the performance benefits of the network twin module. RS utilizes the network twin module but selects actions randomly for strategy construction. GS also uses the network twin module but employs a greedy approach in strategy construction. By comparing PPCA with RS and GS, we can highlight the effectivenesss of the congestion avoidance module.

B. Simulation Results

1) Comparison of Prediction Accuracy: To evaluate prediction accuracy, we compare three metrics: root mean square error (RMSE), mean absolute percentage error (MAPE), and mean absolute error (MAE). RMSE and MAE are absolute error metrics, where RMSE is more sensitive to large flow deviations. MAPE is a relative error metric that becomes more sensitive for small traffic values. Moreover, since the replay traffic in this paper comes from backbone switches, there is a significant difference between peak and idle hours [85]. As shown in Table VI, PPCA achieves the best performance across all three metrics, followed closely by TCN+LSTM and AttCNN. In contrast, models such as ResLSTM, TCN, LSTM, CNN, CNN+LSTM, LSTM+GCN, and GCN exhibit significantly higher errors, indicating weaker predictive capabilities.

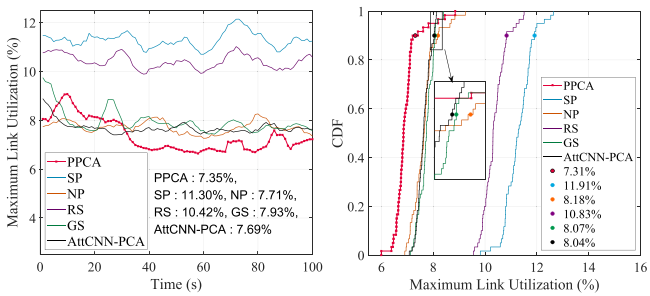
TABLE VI
COMPARISON OF PREDICTION ACCURACY

	RMSE	MAE	MAPE
PPCA	0.02158	0.01536	0.03570
TCN+LSTM	0.02236	0.01611	0.03666
AttCNN	0.02200	0.01634	0.03828
ResLSTM	0.04207	0.03485	0.07602
Baseline TCN	0.06723	0.05411	0.12212
LSTM	0.13704	0.13382	0.30018
CNN	0.33770	0.25469	0.57561
CNN+LSTM	0.33907	0.26618	0.60181
LSTM+GCN	0.33925	0.26773	0.60590
GCN	0.34425	0.26933	0.60650

The superior performance of PPCA comes from the design of customized TCN models and adjacency matrix inputs. This design enables PPCA to better capture localized spatial dependencies more effectively while maintaining TCN's strength in modeling long-term temporal patterns. TCN+LSTM integrates LSTM's sequential modeling capability, enabling effective handling of both short-term fluctuations and long-term trends. AttCNN, which leverages attention mechanisms to enhance feature extraction for bursty traffic, achieves competitive RMSE and MAE but struggles with MAPE due to its weaker performance on stable low-traffic periods. ResLSTM benefits from deep feature extraction but suffers from higher complexity, which may introduce additional prediction variance. The baseline TCN effectively captures long-term dependencies, making it suitable for modeling periodic traffic fluctuations. The traffic of the backbone switch exhibits a long-tailed distribution, with most time steps showing low traffic and a few having extreme peaks. The baseline TCN, trained to predict the entire traffic matrix, tends to smooth out such details, whereas models with attention mechanisms (e.g., AttCNN) or hybrid structures (e.g., TCN+LSTM) can better highlight critical variations. Furthermore, since TCN shares convolution kernel parameters across all time steps, it is well-suited for capturing global periodic patterns but struggles to adapt to highly uneven traffic dynamics. LSTM exhibits high RMSE and MAPE, suggesting that although it captures periodic low-traffic patterns, it fails to learn spatial dependencies, which are crucial in traffic prediction. In contrast, GCN-based models such as LSTM+GCN and GCN perform poorly in this task. While GCN is effective in modeling spatial relationships between network nodes, it struggles with temporal dependencies, leading to large prediction errors. Overall, the results demonstrate that PPCA's design outperforms the baseline TCN and other deep learning models.

2) Comparison of Network Maximum Link Utilization: We compare the performance of several algorithms in terms of

Authorized licensed use limited to: Universitas Indonesia. Downloaded on October 03,2026 at 15:35:47 UTC from IEEE Xplore. Restrictions apply.



(a) 100s of MLU records. (b) CDF of MLU in steady state

Fig. 7. MLU over 100 seconds under different algorithms.

network link utilization, specifically analyzing their variations in maximum link utilization (MLU) across the network over a 100-second simulation period. Fig. 7(a) shows the changes in MLU over time, with annotations indicating the average MLU for each algorithm. Compared to SP, NP, RS, GS, and AttCNN-PCA, PPCA reduces MLU by 3.95%, 0.36%, 3.07%, 0.58%, and 0.34%, respectively. Fig. 7(b) shows the cumulative distribution function (CDF) of MLU in the stable state, with annotations indicating the MLU at the 90th percentile for each algorithm. It can be seen that PPCA has the lowest 90th percentile, which is 4.60%, 0.87%, 3.52%, 0.76%, and 0.73% lower than SP, NP, RS, GS, and AttCNN-PCA, respectively.

As shown in Fig. 7(a), PPCA initially shows a higher MLU but surpasses the other algorithms after 30 seconds. This performance difference can be attributed to the network twin module, which requires waiting for 30 seconds of traffic data before making effective decisions. Around the 25 s, GS, RS, and SP experience peaks, indicating the presence of burst traffic in the network. In contrast, AttCNN-PCA, NP, and PPCA show stable or even decreasing MLU during this time. This observation suggests that these three algorithms are more effective at managing burst traffic compared to GS, RS, and SP. Although NP uses measured traffic information, its congestion avoidance module still generates strategies based on empirical information, further emphasizing the performance advantages of the proposed scheduling algorithm. Additionally, both PPCA and NP employ the same scheduling algorithm, and the comparison between them highlights the benefits of using the network twin module for managing network traffic.

3) Comparison of Bursty Traffic Handling Capability: We conduct a statistical analysis of switch egress traffic data to evaluate PPCA's capability in handling bursty traffic. Specifically, we record the number of packets transmitted per second from switches to hosts under different algorithms. Fig. 8(a) illustrates the peak traffic volume through the switch per second, along with the average values of the peak traffic volume. Compared to SP, NP, RS, GS, and AttCNN-PCA, PPCA reduces the peak packet count by 5,103, 3,599, 1,521, 5,495, and 6,205, respectively, demonstrating its clear advantage in suppressing traffic surges. Fig. 8(b) presents the CDF of the peak traffic volume, with the 90th percentile highlighted. PPCA lowers the 90th percentile by 4,838, 4,357, 698, 6,797, and 6,373 compared with the five baselines, indicating its effectiveness in balancing network traffic and avoiding network congestion. As shown in Fig. 8(a), sudden spikes occur around 13 s and 40 s, reflecting the presence of bursty traffic in the network. Even under these conditions, PPCA maintains low peak levels, demonstrating its robustness in managing bursty traffic.

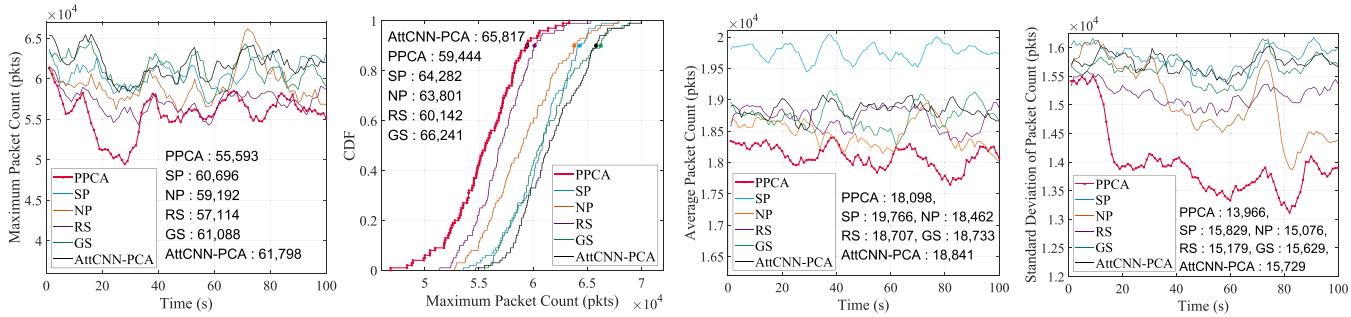
Based on these observations, we further plot Fig. 8(c) and Fig. 8(d). Fig. 8(c) shows the average traffic volume across all switches, where PPCA achieves a lower volume. This result is consistent with the reduced MLU observed in Section V-B2, highlighting its efficiency in conserving bandwidth resources. Fig. 8(d) presents the standard deviation of the traffic volume across all switches, reflecting the balance of traffic distribution. A higher standard deviation indicates that certain nodes experience significantly heavier traffic than others, which may result in localized congestion and unbalanced utilization of network resources. Conversely, a lower standard deviation indicates a more balanced network load. The figure demonstrates that PPCA consistently achieves lower standard deviation values, indicating its effectiveness in balancing network loads and avoiding excessive pressure on individual switches.

In conclusion, by jointly considering peak, average, and standard deviation metrics, PPCA not only suppresses burst peaks effectively but also improves bandwidth efficiency and maintains a balanced network load, demonstrating clear advantages in managing bursty traffic.

4) Comparison of Maximum Queue Depth: We analyze the switch queue depth under different algorithms, as queue occupancy is a key indicator of congestion. By comparing the maximum queue depth (MQD), we can evaluate each algorithm's ability to mitigate congestion under varying traffic loads. Fig. 9(a) shows the MQD of the switches during the simulation, with annotations indicating the standard deviation of the queue depths. Fig. 9(b) presents the CDF of MQD, highlighting the 90th percentile for each algorithm. The results reveal that most algorithms experience a sharp increase in queue depth when subjected to heavy or bursty traffic, reflecting poor capability in mitigating sudden traffic surges and a higher risk of congestion. In contrast, PPCA maintains relatively low queue depths even under significant fluctuations, demonstrating clear advantages in congestion avoidance. Although PPCA does not achieve the lowest standard deviation—due to larger fluctuations in the initial adaptation phase—it stabilizes quickly and maintains the lowest MQD thereafter. These results highlight PPCA's robustness in handling dynamic traffic conditions and avoiding congestion.

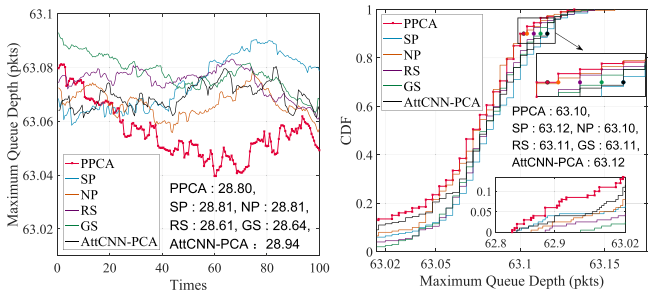
5) Comparison of Host Packet Transmissions: We conducted a statistical analysis of packet transmissions at the host, with a particular focus on the average number of packets per second (PPS) and bits per second (BPS) across different algorithms. Since the simulation is based on real network traffic datasets, fluctuations are observed across all algorithms. As shown in Fig. 10, PPCA achieves an average PPS of 3,139, higher than all other baselines. In terms of BPS, PPCA also shows a modest improvement, reaching 182,823 bps, indicating more efficient bandwidth utilization. Although the differences in PPS and BPS among these algorithms are not significant, the strength of PPCA lies in its ability to adapt to traffic fluctuations and maintain stable transmission. Together with the earlier results

Authorized licensed use limited to: Universitas Indonesia. Downloaded on October 03,2026 at 15:35:47 UTC from IEEE Xplore. Restrictions apply.



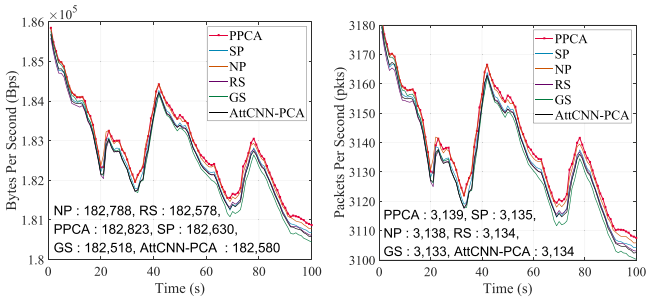
(a) The peak traffic volume through the switch. (b) CDF of the peak traffic volume. (c) Average traffic volume across all switches. (d) Standard deviation of the traffic volume across all switches.

Fig. 8. The statistical analysis of switch egress traffic data under different algorithms.



(a) 100s of MQD records. (b) CDF of MQD.

Fig. 9. MQD under different algorithms.

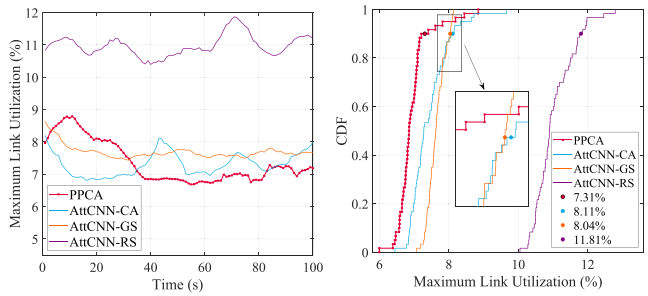


(a) Bytes per second. (b) Packets per second.

Fig. 10. Host transmission performance under different algorithms.

on queue depth and MLU, these findings show that PPCA not only excels in congestion avoidance and traffic management but also provides a slight advantage in transmission performance.

6) Comparison of MLU Under Different Module Combinations: We conduct supplementary simulations to further validate the performance of the proposed method. Specifically, we use AttCNN as the prediction model and combine it with RS, GS, and the congestion avoidance module proposed in this paper. The three variants are denoted as AttCNN-RS, AttCNN-GS, and AttCNN-CA, respectively. We evaluate the MLU of each variant over 100 seconds, as shown in Fig. 11(a). It can be observed that with the same prediction model, the congestion avoidance module designed in this paper demonstrates a lower value. However, it still falls short compared to PPCA. Fig. 11(b) presents the CDF of MLU in the steady state, with annotations indicating



(a) MLU under different module combinations. (b) CDF of MLU under different module combinations in steady state.

Fig. 11. Comparison of MLU under different module combinations.

the 90th percentile for each algorithm. Compared to AttCNN-GS, AttCNN-CA exhibits a higher 90th percentile, indicating instability in the congestion avoidance module. Nevertheless, this disadvantage can be effectively mitigated by integrating the network twin module.

VI. Conclusion

In this paper, we propose a congestion avoidance mechanism called PPCA for DCI. First, PPCA adopts a neighbor-assisted traffic prediction method, building customized models for each switch using neighbor information to achieve more accurate predictions. Second, it leverages these predictions for flow scheduling. By parallel strategy combination and assessment, PPCA quickly identifies the optimal congestion avoidance strategy, improving overall network performance. Simulations with real-world traffic replay demonstrate that PPCA effectively mitigates congestion, balances network traffic, and reduces transmission queuing. It also efficiently handles burst traffic, making it well-suited for large-scale data synchronization and parameter exchange. In future work, we will explore the integration of PPCA with various congestion control algorithms to enable efficient and comprehensive end-to-network collaborative transmission.

References

[1] R. Wang et al., "Joint energy and computation workload management for geo-distributed data centers," *IEEE Trans. Green Commun. Netw.*, vol. 9. no. 4, pp. 2115–2128, Dec. 2025.

Authorized licensed use limited to: Universitas Indonesia. Downloaded on October 03,2026 at 15:35:47 UTC from IEEE Xplore. Restrictions apply.

[2] S. Chen, J. Li, Q. Yuan, H. He, S. Li, and J. Yang, "Two-timescale joint optimization of task scheduling and resource scaling in multi-data center system based on multi-agent deep reinforcement learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 12, pp. 2331–2346, Dec. 2024.

[3] S. Ha, I. Rhee, and L. Xu, "CUBIC: A new TCP-friendly high-speed TCP variant," *SIGOPS Operating Syst. Rev.*, vol. 42, no. 5, pp. 64–74, 2008.

[4] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "BBR: Congestion-based congestion control," *Commun. ACM*, vol. 60, no. 2, pp. 58–66, 2017.

[5] K. Ramakrishnan, S. Floyd, and D. Black, *The Addition of Explicit Congestion Notification (ECN) to IP*, RFC 3168, Internet Engineering Task Force, Fremont, CA, USA, 2001. [Online]. Available: https://www.rfc-editor.org/rfc/rfc3168.html

[6] J. Chen et al., "SDATP: An SDN-based traffic-adaptive and service-oriented transmission protocol," *IEEE Trans. Cogn. Commun. Netw.*, vol. 6, no. 2, pp. 756–770, Jun. 2020.

[7] K. Liu et al., "INCC: In-network congestion control with proactive bottleneck awareness," *IEEE/ACM Trans. Netw.*, early access, Jul. 29, 2025, doi: 10.1109/TON.2025.3589553.

[8] S. Wang et al., "Predictable vfabric on informative data plane," in *Proc. ACM SIGCOMM*, 2022, pp. 615–632.

[9] Y. Li et al., "HPCC: High precision congestion control," in *Proc. ACM SIGCOMM*, 2019, pp. 44–58.

[10] Z. Xu et al., "DOFMS: DRL-based out-of-order friendly multipath scheduling in mobile heterogeneous networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 8, pp. 8274–8288, Aug. 2024.

[11] N. C. Luong et al., "Applications of deep reinforcement learning in communications and networking: A survey," *IEEE Commun. Surv. Tut.*, vol. 21, no. 4, pp. 3133–3174, Fourthquarter 2019.

[12] R. Zhang, K. Xiong, Y. Lu, P. Fan, D. W. K. Ng, and K. B. Letaief, "Energy efficiency maximization in RIS-assisted SWIPT networks with RSMA: A PPO-based approach," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 5, pp. 1413–1430, May 2023.

[13] J. Zhou et al., "Antelope: A framework for dynamic selection of congestion control algorithms," in *Proc. IEEE Int. Conf. Netw. Protoc.*, 2021, pp. 1–11.

[14] C. Ren, S. Bai, Y. Wang, and Y. Li, "Achieving near-optimal traffic engineering using a distributed algorithm in hybrid SDN," *IEEE Access*, vol. 8, pp. 29111–29124, 2020.

[15] Y. Wu, K. Zhang, and Y. Zhang, "Digital twin networks: A survey," *IEEE Internet Things J.*, vol. 8, no. 18, pp. 13789–13804, Sep. 2021.

[16] P. Almasan et al., "Network digital twin: Context, enabling technologies, and opportunities," *IEEE Commun. Mag.*, vol. 60, no. 11, pp. 22–27, Nov. 2022.

[17] L. Hui, M. Wang, L. Zhang, L. Lu, and Y. Cui, "Digital twin for networking: A data-driven performance modeling perspective," *IEEE Netw.*, vol. 37, no. 3, pp. 202–209, May/Jun. 2023.

[18] A. Azzouni and G. Pujolle, "NeuTM: A neural network-based framework for traffic matrix prediction in SDN," in *Proc. IEEE/IFIP Netw. Operations Manage. Symp.*, 2018, pp. 1–5.

[19] A. Azari, P. Papapetrou, S. Denic, and G. Peters, "Cellular traffic prediction and classification: A comparative evaluation of LSTM and arima," in *Proc. Int. Conf. Discov. Sci.*, 2019, pp. 129–144.

[20] S. Zeb, M. A. Rathore, A. Mahmood, S. A. Hassan, J. Kim, and M. Gidlund, "Edge intelligence in softwarized 6G: Deep learning-enabled network traffic predictions," in *Proc. IEEE GLOBECOM Workshops*, 2021, pp. 1–6.

[21] I. Alawe, A. Ksentini, Y. Hadjadj-Aoul, and P. Bertin, "Improving traffic forecasting for 5G core network scalability: A machine learning approach," *IEEE Netw.*, vol. 32, no. 6, pp. 42–49, Nov./Dec. 2018.

[22] Y. Hua, Z. Zhao, R. Li, X. Chen, Z. Liu, and H. Zhang, "Deep learning with long short-term memory for time series prediction," *IEEE Commun. Mag.*, vol. 57, no. 6, pp. 114–119, Jun. 2019.

[23] K. Gao et al., "Predicting traffic demand matrix by considering inter-flow correlations," in *Proc. IEEE INFOCOM 2020 - IEEE Conf. Comput. Commun. Workshops*, 2020, pp. 165–170.

[24] Z. Wang and V. W. S. Wong, "Cellular traffic prediction using deep convolutional neural network with attention mechanism," in *Proc. IEEE Int. Conf. Commun.*, 2022, pp. 2339–2344.

[25] M. Li, Y. Wang, Z. Wang, and H. Zheng, "A deep learning method based on an attention mechanism for wireless network traffic prediction," *Ad Hoc Netw.*, vol. 107, 2020, Art. no. 102258.

[26] J. Bi, X. Zhang, H. Yuan, J. Zhang, and M. Zhou, "A hybrid prediction method for realistic network traffic with temporal convolutional network and LSTM," *IEEE Trans. Autom. Sci. Eng.*, vol. 19, no. 3, pp. 1869–1879, Jul. 2022.

[27] Q. Liu, J. Li, and Z. Lu, "ST-tran: Spatial-temporal transformer for cellular traffic prediction," *IEEE Commun. Lett.*, vol. 25, no. 10, pp. 3325–3329, Oct. 2021.

[28] Y. Hu, Y. Zhou, J. Song, L. Xu, and X. Zhou, "Citywide mobile traffic forecasting using spatial-temporal downsampling transformer neural networks," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 1, pp. 152–165, Mar. 2023.

[29] W. Shen, H. Zhang, S. Guo, and C. Zhang, "Time-wise attention aided convolutional neural network for data-driven cellular traffic prediction," *IEEE Commun. Lett.*, vol. 10, no. 8, pp. 1747–1751, Aug. 2021.

[30] J. Zhou et al., "Multiscale network traffic prediction method based on deep echo-state network for Internet of Things," *IEEE Internet Things J.*, vol. 9, no. 21, pp. 21862–21874, Nov. 2022.

[31] H. Ma, K. Yang, and Y. Jiao, "Cellular traffic prediction via byzantine-robust asynchronous federated learning," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 4, pp. 2402–2414, Jul./Aug. 2025.

[32] L. Nie et al., "Network traffic prediction in industrial Internet of Things backbone networks: A multitask learning mechanism," *IEEE Trans. Ind. Informat.*, vol. 17, no. 10, pp. 7123–7132, Oct. 2021.

[33] Z. Wang et al., "Large-scale measurements and prediction of DC-WAN traffic," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 5, pp. 1390–1405, May 2023.

[34] J. Chen, X. Wang, and X. Xu, "GC-LSTM: Graph convolution embedded LSTM for dynamic network link prediction," *Appl. Intell.*, vol. 52, no. 7, pp. 7513–7528, 2022.

[35] R. Xie et al., "M ^{2} STL: Multi-range multi-level spatial-temporal learning model for network traffic prediction," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 5, pp. 4315–4329, Sep./Oct. 2024.

[36] S. Zou, J. Huang, J. Liu, T. Zhang, N. Jiang, and J. Wang, "GTCP: Hybrid congestion control for cross-datacenter networks," in *Proc. IEEE Int. Conf. Distrib. Comput. Syst.*, 2021, pp. 932–942.

[37] G. Zeng et al., "Congestion control for cross-datacenter networks," *IEEE/ACM Trans. Netw.*, vol. 30, no. 5, pp. 2074–2089, Oct. 2022.

[38] Y. Geng et al., "Delay based congestion control for cross-datacenter networks," in *Proc. IEEE/ACM Int. Symp. Qual. Service*, 2023, pp. 1–4.

[39] M. Long, J. Han, W. Wang, J. Yang, and K. Xue, "LSCC: Link-segmented congestion control for RDMA in cross-datacenter networks," in *Proc. IEEE/ACM Int. Symp. Qual. Service*, 2024, pp. 1–10.

[40] G. Zeng, J. Qiu, Y. Yuan, H. Liu, and K. Chen, "FlashPass: Proactive congestion control for shallow-buffered WAN," in *Proc. IEEE Int. Conf. Netw. Protoc.*, 2021, pp. 1–12.

[41] W. Quan, M. Liu, N. Cheng, X. Zhang, D. Gao, and H. Zhang, "Cybertwin-driven DRL-based adaptive transmission scheduling for software defined vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 5, pp. 4607–4619, May 2022.

[42] Y.-R. Chen, A. Rezapour, W.-G. Tzeng, and S.-C. Tsai, "RL-routing: An SDN routing algorithm based on deep reinforcement learning," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 4, pp. 3185–3199, Oct.–Dec. 2020.

[43] P. Almasan, S. Xiao, X. Cheng, X. Shi, P. Barlet-Ros, and A. Cabellos-Aparicio, "ENERO: Efficient real-time WAN routing optimization with deep reinforcement learning," *Comput. Netw.*, vol. 214, 2022, Art. no. 109166.

[44] J. Chen et al., "ASTPPO: A proximal policy optimization algorithm based on the attention mechanism and spatio-temporal correlation for routing optimization in software-defined networking," *Peer-to-Peer Netw. Appl.*, vol. 16, no. 5, pp. 2039–2057, 2023.

[45] K. Liu et al., "Reliable PPO-based concurrent multipath transfer for time-sensitive applications," *IEEE Trans. Veh. Technol.*, vol. 72, no. 10, pp. 13575–13590, Oct. 2023.

[46] S. Troia, F. Sapienza, L. Varé, and G. Maier, "On deep reinforcement learning for traffic engineering in SD-WAN," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 7, pp. 2198–2212, Jul. 2021.

[47] J. Chen et al., "ALBRL: Automatic load-balancing architecture based on reinforcement learning in software-defined networking," *Wireless Commun. Mobile Comput.*, vol. 2022, no. 1, 2022, Art. no. 3866143.

[48] K. Liu et al., "Deadline-constrained multi-agent collaborative transmission for delay-sensitive applications," *IEEE Trans. Cogn. Commun. Netw.*, vol. 9, no. 5, pp. 1370–1384, Oct. 2023.

[49] X. Liu, S. Zhao, and X. Wang, "Geminet: Learning the duality-based iterative process for lightweight traffic engineering in changing topologies," 2025, *arXiv:2506.23640*.

[50] R. Zhang et al., "Generative AI for space-air-ground integrated networks," *IEEE Wireless Commun.*, vol. 31, no. 6, pp. 10–20, Dec. 2024.

Authorized licensed use limited to: Universitas Indonesia. Downloaded on October 03,2026 at 15:35:47 UTC from IEEE Xplore. Restrictions apply.

[51] J. Zhou et al., "A machine learning-based framework for dynamic selection of congestion control algorithms," *IEEE/ACM Trans. Netw.*, vol. 31, no. 4, pp. 1566–1581, Aug. 2023.
[52] X. Liu, S. Zhao, Y. Cui, and X. Wang, "FIGRET: Fine-grained robustness-enhanced traffic engineering," in *Proc. ACM SIGCOMM*, 2024, pp. 117–135.
[53] Y. Perry et al., "Dote: Rethinking (predictive) WAN traffic engineering," in *Proc. USENIX Symp. Networked Syst. Des. Implementation*, 2023, pp. 1557–1581.
[54] Y. Liu et al., "Adaptive and low-cost traffic engineering: A traffic matrix clustering perspective," *IEEE J. Sel. Areas Commun.*, vol. 43, no. 2, pp. 510–523, Feb. 2025.
[55] N. Geng, M. Xu, Y. Yang, E. Dong, and C. Liu, "Adaptive and low-cost traffic engineering based on traffic matrix classification," in *Proc. IEEE Int. Conf. Comput. Commun. Netw.*, 2020, pp. 1–9.
[56] C. Ren, S. Wang, J. Ren, X. Wang, T. Song, and D. Zhang, "Enhancing traffic engineering performance and flow manageability in hybrid SDN," in *Proc. IEEE IEEE Glob. Commun. Conf.*, 2016, pp. 1–7.
[57] H. Zhang et al., "Boosting bandwidth availability over inter-DC WAN," in *Proc. ACM Int. Conf. Emerg. Netw. EXperiments Technol.*, 2021, pp. 297–312.
[58] H. Zhang et al., "Achieving high availability in inter-DC WAN traffic engineering," *IEEE/ACM Trans. Netw.*, vol. 31, no. 6, pp. 2406–2421, Dec. 2023.
[59] J. Chen, W. Xiao, H. Zhang, J. Zuo, and X. Li, "Dynamic routing optimization in software-defined networking based on a metaheuristic algorithm," *J. Cloud Comput.*, vol. 13, no. 1, p. 41, 2024.
[60] M. Chen, R. Li, Z. Zhao, and H. Zhang, "RAN information-assisted TCP congestion control via DRL with reward redistribution," in *Proc. IEEE Int. Conf. Commun.*, 2021, pp. 1–7.
[61] M. Chen, R. Li, J. Crowcroft, J. Wu, Z. Zhao, and H. Zhang, "RAN information-assisted TCP congestion control using deep reinforcement learning with reward redistribution," *IEEE Trans. Commun.*, vol. 70, no. 1, pp. 215–230, Jan. 2022.
[62] H. Yuan, J. Bi, S. Li, J. Zhang, and M. Zhou, "An improved LSTM-based prediction approach for resources and workload in large-scale data centers," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 22816–22829, Jun. 2024.
[63] M. Kumar, A. Kishor, J. K. Samariya, and A. Y. Zomaya, "An autonomic workload prediction and resource allocation framework for fog-enabled industrial IoT," *IEEE Internet Things J.*, vol. 10, no. 11, pp. 9513–9522, Jun. 2023.
[64] Y. Liu, J. Zhang, W. Li, Q. Wu, and P. Li, "Load balancing oriented predictive routing algorithm for data center networks," *Future Internet*, vol. 13, no. 2, 2021.
[65] G. Diel, C. C. Miers, M. A. Pillon, and G. P. Koslovski, "Data classification and reinforcement learning to avoid congestion on SDN-based data centers," in *Proc. IEEE GLOBECOM 2022-2022 IEEE Glob. Commun. Conf.*, 2022, pp. 2861–2866.
[66] D. Gustavo, C. M. Charles, A. P. Maurıćio, and P. K. Guilherme, "RSCAT: Towards zero touch congestion control based on actor–critic reinforcement learning and software-defined networking," *J. Netw. Comput. Appl.*, vol. 215, 2023, Art. no. 103639.
[67] B. Li, L. Chen, Z. Yang, and H. Xiang, "preDQN-based TAS traffic scheduling in intelligence endogenous networks," *IEEE Syst. J.*, vol. 18, no. 2, pp. 997–1008, Jun. 2024.
[68] A. Yu, Q. Yang, L. Dou, and M. Cheriet, "Federated imitation learning: A cross-domain knowledge sharing framework for traffic scheduling in 6G ubiquitous IoT," *IEEE Netw.*, vol. 35, no. 5, pp. 136–142, Sep./Oct. 2021.
[69] Y. Hao, F. Li, C. Zhao, and S. Yang, "Delay-oriented scheduling in 5G downlink wireless networks based on reinforcement learning with partial observations," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 380–394, Feb. 2023.
[70] A. Kumari and A. S. Sairam, "Look ahead before you leap: SDN switch migration scheduling for load balancing," *Wireless Pers. Commun.*, pp. 1–30, 2024.
[71] M. Hao, X. Sun, Y. Li, and H. Zhang, "Edge-side cellular network traffic prediction based on trend graph characterization network," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 6, pp. 6118–6129, Nov./Dec. 2024.
[72] R. Zhang et al., "Interactive AI with retrieval-augmented generation for next generation networking," *IEEE Netw.*, vol. 38, no. 6, pp. 414–424, Nov. 2024.
[73] J. Lai et al., "Deep learning based traffic prediction method for digital twin network," *Cogn. Computation*, vol. 15, no. 5, pp. 1748–1766, 2023.
[74] M. A. Qureshi et al., "PLB: Congestion signals are simple and effective for network load balancing," in *Proc. ACM SIGCOMM*, 2022, pp. 207–218.
[75] K.-C. Tsai, L. Fan, L.-C. Wang, R. Lent, and Z. Han, "Multi-commodity flow routing for large-scale leo satellite networks using deep reinforcement learning," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2022, pp. 626–631.
[76] C. Chekuri, S. Khanna, and F. B. Shepherd, "Multicommodity flow, well-linked terminals, and routing problems," in *Proc. ACM Symp. Theory Comput.*, 2005, pp. 183–192.
[77] Z. Xu et al., "Harmonypath: Fine-grained flexible multipath transmission for mobile differentiated services," *IEEE Trans. Mobile Comput.*, vol. 24, no. 8, pp. 7312–7327, Aug. 2025.
[78] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," 2018, *arXiv:1803.01271*.
[79] T. Gong, S. Zhang, M. Xu, Z. Li, and O. A. Dobre, "Lightweight TCN-based spatial-temporal channel extrapolation for RIS-aided communication," *IEEE Trans. Veh. Technol.*, vol. 73, no. 8, pp. 12219–12224, Aug. 2024.
[80] X. Jin et al., "Dynamic scheduling of network updates," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 4, pp. 539–550, 2014.
[81] MAWI Working Group, "MAWI working group traffic archive," Accessed: Mar. 23, 2025. [Online]. Available: https://mawi.wide.ad.jp/mawi/
[82] Z. Wang, Z. Li, G. Liu, Y. Chen, Q. Wu, and G. Cheng, "Examination of WAN traffic characteristics in a large-scale data center network," in *Proc. ACM Internet Meas. Conf.*, 2021, pp. 1–14.
[83] T. Luo et al., "Proactive congestion avoidance mechanism with attention based CNN," in *Proc. IEEE/CIC Int. Conf. Commun. China Workshops*, 2023, pp. 1–6.
[84] J. Zhang, F. Chen, Z. Cui, Y. Guo, and Y. Zhu, "Deep learning architecture for short-term passenger flow forecasting in urban rail transit," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 11, pp. 7004–7014, Nov. 2021.
[85] X. Zhang et al., "Towards green cloud transmission: Observation from practical inter-cloud links," in *Proc. IEEE GLOBECOM 2023-2023 IEEE Glob. Commun. Conf.*, 2023, pp. 5877–5882.



Xue Zhang received the master's degree in electronic and communication engineering from Beijing Jiaotong University, Beijing, China, in 2019. She is currently working toward the Ph.D. degree with the School of Electronic and Information Engineering, Beijing Jiaotong University. Her current research interests include congestion control, software-defined networking, network function virtualization, and programmable network languages.



Yuming Zhang (Member, lEEE) received the Ph.D. degree in telecommunications and information system from Beijing Jiaotong University, Beijing, China, in 2021. He was a Postdoctor with the School of Computer Science, Beijing University of Posts and Telecommunications, Beijing, from 2021 to 2024. He is currently a Lecturer with the School of Electronic and Information Engineer, Beijing Jiaotong University. His current research interests include drone network, mobile edge computing, queue theory, and machine learning.



Wei Quan (Senior Member, lEEE) received the Ph.D. degree in communication and information system from the Beijing University of Posts and Telecommunications, Beijing, China, in 2014. He is currently a Full Professor with the School of Electronic and Information Engineering, Beijing Jiaotong University, Beijing. He has coauthored more than 50 papers in prestigious international journals and conferences, including IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, *IEEE Communications Magazine*, IEEE WIRELESS COMMUNICATIONS, and IEEE NETWORK. His research interests include reliable transmission in mobile networks and vehicular networks.

Authorized licensed use limited to: Universitas Indonesia. Downloaded on October 03,2026 at 15:35:47 UTC from IEEE Xplore. Restrictions apply.



Mingyuan Liu (Member, lEEE) received the Ph.D. degree in information and communication engineering from Beijing Jiaotong University, Beijing, China, in 2023. He is currently a Lecturer with the School of Electronic and Information Engineering, Beijing Jiaotong University. His research interests include future networks, software defined networking, and Internet of Things.



Yirong Zhuang received the master's degree from Sun Yat-sen University, Guangzhou, China, in 2001. He is currently a Senior Engineer with China Telecom Research Institute, Guangzhou. His current research interests include internet protocol network, content delivery network, and internet protocol television.



Yan Luo received the bachelor's degree in data science and Big Data technology from the China University of Petroleum, Karamay, China, in 2022. He is currently working toward the Ph.D. degree with the School of Electronic and Information Engineering, Beijing Jiaotong University, Beijing, China. His research interests include multipath transfer, software-defined networking, machine learning, and space-air-ground integrated network.



Tong Luo received the master's degree from Beijing Jiaotong University, Beijing, China, in 2023. He is currently a staff Member with China Mobile (Suzhou) Software Technology Corporation, Suzhou, China. His research interests include cloud computing and software defined networking.



Jie Liang is an expert in the China Telecom Internet and Application Technology Commission, Senior expert in China Telecom Internet Protocol (IP) network. She was once responsible for the technology research and network planning of China Telecom IP network and IP television system. She has 20 years of working experience in the communication industry. Her current research interests include video technology, multicast streaming, and autonomous network.



Kang Liu received the Ph.D. degree in communication and information system from Beijing Jiaotong University, Beijing, China, in 2023. He is currently a Postdoctor with the Peng Cheng Laboratory, Department of New Networks, Shenzhen, China. He has authored or coauthored more than 10 papers in prestigious international journals and conferences, including IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING, IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, IEEE ICC, and IEEE INFOCOM WKSHPS. His research interests include concurrent multipath transfer, machine learning, software-defined networking, and programmable data plane.



Hongke Zhang received the Ph.D. degree in communication and information system from the University of Electronic Science and Technology of China, Chengdu, China, in 1992. He is currently a Professor with the School of Electronic and Information Engineering, Beijing Jiaotong University, Beijing, China, where he currently directs the National Engineering Center of China on Mobile Specialized Network. He is also an Academician of China Engineering Academy. His current research interests include architecture and protocol design for the future Internet and specialized networks. He is an Associate Editor for IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT and IEEE INTERNET OF THINGS JOURNAL.

Authorized licensed use limited to: Universitas Indonesia. Downloaded on October 03,2026 at 15:35:47 UTC from IEEE Xplore. Restrictions apply.